Toward High-Precision Target-Free LiDAR-Camera Extrinsic Calibration: Multi-Modal Geometric Edge Matching and Visibility-Guided Optimization

Baosheng Zhang, Lin Zhang, *Senior Member, IEEE*, Shengjie Zhao, *Senior Member, IEEE*, and Yicong Zhou, *Senior Member, IEEE*

***Abstract*—The rapid advancement of autonomous driving and embodied intelligence highlights the critical need for robust multi-sensor systems. In such systems, LiDAR and cameras are frequently paired—leveraging their complementary strengths: LiDAR excels at high-precision measurements, while cameras deliver rich texture information. Accurate extrinsic calibration between these two sensors is a fundamental prerequisite for effective data fusion. However, traditional calibration methods face challenges in extracting robust multi-modal features under complex scenarios and typically overlook feature visibility check during optimization. To address these limitations, we propose a novel target-free method for the automatic extrinsic calibration of LiDAR and camera. Our approach incorporates key innovations in both feature extraction and pose optimization. For LiDAR point clouds, we design a feature detector based on local eigenvalue decomposition to reliably identify geometric edge points. For images, we overcome the limitations of traditional line feature extraction methods, which are susceptible to texture interference, by fusing raw images with depth maps estimated via Depth Anything V2, enabling robust extraction of geometric edge lines. Furthermore, to mitigate the impact of invisible features, a step often ignored in existing methods, we integrate a visibility check module into our optimization framework. Finally, LiDAR-camera extrinsic calibration is achieved by establishing correspondences between the extracted multi-modal geometric edge features and minimizing the associated constraints. Comprehensive experiments across diverse scenes show that our method consistently outperforms state-of-the-art (SOTA) counterparts. Notably, even in the worst-case scene, it achieves rotational errors below 0.86° and translational errors below 0.09 meters, outperforming the top existing methods by a substantial margin. For full reproducibility, our code is publicly accessible at https://cslinzhang.github.io/CalibrationLC**

***Index Terms*—LiDAR-camera extrinsic calibration, automatic target-free calibration, multi-sensor system, feature extraction.**

Received 29 October 2025; revised 2 April 2026, 7 May 2026, and 15 June 2026; accepted 28 August 2026. Date of current version 14 September 2026. This work was supported in part by the National Natural Science Foundation of China under Grant 62476202, Grant 62272343, and Grant 62672348; in part by the Explorers Program of Shanghai (Basic Research Funding) from Shanghai Municipal Science and Technology Commission under Grant 26TS1403800; and in part by the Fundamental Research Funds for the Central Universities. The associate editor coordinating the review of this article and approving it for publication was Prof. Ravibabu Mulaveesala. *(Corresponding author: Lin Zhang.)*

Baosheng Zhang, Lin Zhang, and Shengjie Zhao are with the School of Computer Science and Technology, Tongji University, Shanghai 201804, China (e-mail: baoshengzhang@tongji.edu.cn; cslinzhang@tongji.edu.cn; shengjiezhao@tongji.edu.cn).

Yicong Zhou is with the Department of Computer and Information Science, University of Macau, Macau, China (e-mail: yicongzhou@um.edu.mo).

Digital Object Identifier 10.1109/TIP.2026.3730786

I. Introduction

IN FIELDS such as autonomous driving and embodied intelligence, LiDAR and camera are widely deployed for their respective advantages in high-precision measurement and rich texture information. However, uncalibrated or misaligned LiDAR-camera systems cause abnormal point cloud-image fusion, leading to target localization deviations, decision errors, and thus posing severe threats to the operational safety of autonomous driving and robot navigation. Therefore, LiDAR-camera extrinsic calibration is of high research value and has been extensively studied in past decade [2], [3], [4].

Existing LiDAR-camera calibration methods can be categorized into two categories: the target-based ones and the target-free ones [5], [6], [7]. Among them, the target-based calibration methods rely on specialized calibration targets, which entail substantial costs and intricate operational workflows. These inherent limitations impose significant constraints on their practical engineering deployment. In contrast, target-free calibration methods eliminate the need for dedicated calibration targets and can directly extract features from environments to complete matching. However, their core bottleneck lies in the difficulty in extracting robust matching features from multi-modal data. Besides, although end-to-end learning-based calibration methods [8], [9] have been successively proposed in recent years, they suffer from defects: insufficient accuracy for high-precision scenarios, heavy reliance on annotated data, and weak generalization, which severely restrict their engineering practicality and deployment efficiency. In addition, a notable oversight in existing calibration frameworks is the lack of feature visibility check step during optimization—a factor crucial for mitigating the impact of occluded points.

As mentioned above, although LiDAR-camera extrinsic calibration has been explored for some decades, there are still two significant challenges persisting in complex environments: **1) the difficulty in extracting robust features from multi-modal data**, and **2) the lack of feature visibility check during optimization**. To overcome these limitations, we propose a novel target-free calibration method built on the following three key innovations.

First, we propose a novel point cloud geometric feature extraction algorithm, leveraging the spatial information inherent in local point cloud structures. Unlike existing point cloud

1941-0042 © 2026 IEEE. All rights reserved, including rights for text and data mining, and training of artificial intelligence and similar technologies. Personal use is permitted, but republication/redistribution requires IEEE permission.
See https://www.ieee.org/publications/rights/index.html for more information.

Authorized licensed use limited to: TONGJI UNIVERSITY. Downloaded on October 18,2026 at 00:47:47 UTC from IEEE Xplore. Restrictions apply.

feature extraction methods, which often perform poorly in complex environments and are vulnerable to interference from LiDAR measurement noise, our algorithm constructs dedicated feature detectors and evaluates feature validity based on local point distributions, enabling robust identification of geometric edge features for subsequent matching. A detailed formulation of this algorithm is presented in Section III-B.1.

Second, a hybrid image geometric feature extraction framework integrating traditional extraction and learning-based depth estimation is proposed. Unlike traditional methods [10], [11]—which identify intensity gradients, making them sensitive to textural noise and prone to detecting non-geometric spurious features—depth images inherently carry geometric information and are robust to surface textures. Therefore, our method first generates a depth image from the raw input, then extracts geometric lines from it, and simultaneously utilizes the gradient information of the depth image to identify and retain the geometric line features extracted from the raw image. This process ensures that only robust geometric edge lines are retained, and further details are provided in Section III-B.2.

Third, we find that different extrinsic parameters correspond to different LiDAR viewpoints, resulting in different measurement results. Consequently, certain points may be occluded during optimization. To maintain calibration accuracy, these occluded points, particularly those invisible geometric edge points, must be identified and removed. Inspired by the efficacy of 3D Gaussian splatting for novel view synthesis, we propose a novel visibility check module based on Gaussian ellipsoid overlapping to address occlusion during optimization. This module evaluates the visibility of the target point $\boldsymbol{p}_i^{\mathcal{L}}$ by analyzing the overlap of Gaussian ellipsoids between $\boldsymbol{p}_i^{\mathcal{L}}$ and its neighboring points $\{\boldsymbol{p}_{ij}^{\mathcal{L}}\}_{j=1}^{n}$, as well as the consistency of the local surface normals at $\boldsymbol{p}_i^{\mathcal{L}}$ and $\{\boldsymbol{p}_{ij}^{\mathcal{L}}\}_{j=1}^{n}$. This method effectively filters out invisible features, thereby improving calibration accuracy. The specifics of this module are detailed in Section III-C.

To verify the effectiveness of our method, Fig. 1 presents a comparison of the calibration results between a SOTA method [1] and our method in both indoor and outdoor scenes. As shown in Fig. 1, our method exhibits superior performance, as reflected in the more accurate colorization of both foreground and background point clouds—an advantage particularly pronounced in the outdoor environment of Scene 2. This performance gain stems from our adoption of geometric edge features and integration of a visibility check module, which collectively ensure that only valid features are involved in optimization.

The subsequent sections are organized as follows. A review of the relevant literature is provided in Section II. Section III elaborates on our proposed framework. We then validate our method in Section IV, presenting extensive qualitative and quantitative results across multiple scenes. Finally, we conclude and discuss future researches in Section V.

II. Related Work

Extrinsic calibration of LiDAR-camera systems is a 2D-3D registration problem of estimating the rigid transformation between the sensors. Existing solutions to this problem can be roughly categorized as target-based ones and target-free ones, and their representatives are briefly reviewed below.

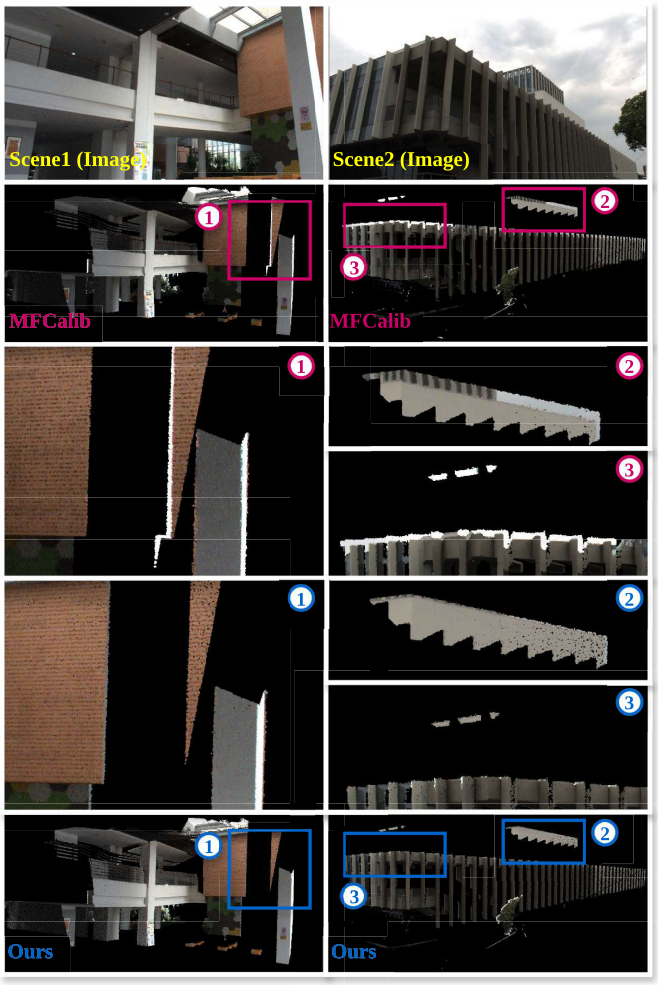


Fig. 1. Qualitative comparison of calibration results in typical indoor and outdoor scenes. The top row shows raw scene images, followed by colorized point clouds from MFCalib [1] and Ours. As highlighted in Scene 1 (region ①) and Scene 2 (regions ② and ③), MFCalib exhibits noticeable projection misalignments and "color bleeding". In contrast, our method ensures pixel-level alignment between point clouds and geometric edges, maintaining distinct boundaries between foreground and background objects.

A. Target-Based Calibration

Target-based calibration methods are widely used for calibrating vision sensors, including various LiDARs and cameras. These approaches involve designing specific checkerboards and identifying detectable features within the full field of view (FoV) of the sensors. The planar checkerboard pattern is the most commonly used target for both intrinsic and extrinsic calibrations of LiDAR and camera [12], [13]. The seminal work by Zhang and Pless [14] introduced a calibration method based on observing a planar checkerboard pattern, which establishes constraints between the "views" captured by the LiDAR and camera. They provided a direct solution that minimizes the algebraic error from those constraints, followed by a nonlinear refinement to minimize the reprojection error. Later, Xie et al. [15] developed a marker pattern named A4LidarTag,

Authorized licensed use limited to: TONGJI UNIVERSITY. Downloaded on October 18,2026 at 00:47:47 UTC from IEEE Xplore. Restrictions apply.

composed of circular holes. This design exploits differences in depth measurements to encode location information. Based on A4LidarTag, they proposed an automatic extrinsic calibration framework for solid-state LiDAR-camera systems. Around the same period, Li et al. [16] proposed another calibration board featuring checkerboard grids and circular holes. Their method enables automatic determination of extrinsic parameters by matching the centers of circular holes extracted from both the image and LiDAR point clouds. In recent advances, Cui et al. [17] proposed an automatic target-based calibration method tailored for recent non-repetitive scanning LiDAR-camera systems. By leveraging the perspective-n-point algorithm, with optimized 3D corners from point clouds and 2D corners from camera images, they successfully determined the extrinsic parameters of LiDAR-camera systems. Expanding further, Wei et al. [18] presented a full-lifecycle calibration method integrating offline and online processes. For offline calibration, they designed a specialized calibration board with circular array holes and surrounding positioning holes. Such a board enables automatic feature detection in both LiDAR and camera modalities, reducing manual intervention. Most recently, Wang et al. [19] proposed LONS-Calib, a framework dedicated to the extrinsic calibration of LiDAR and omnidirectional camera. They optimized chessboard corner extraction for highly distorted images and projected these corners onto a unit sphere for camera pose estimation. A variability-based quality score automates sample selection, ensuring calibration accuracy and generalizability, with typical optimization times under 0.3 seconds on consumer-grade devices.

However, the reliance on dedicated artificial targets limits the applicability of these methods in scenarios where such calibration objects are unavailable. This dependence also makes the LiDAR-camera calibration process more costly and complex.

B. Target-Free Calibration

Target-free calibration methods estimate extrinsic parameters by leveraging inherent environmental features like planes and edges to establish geometric constraints, eliminating the need for artificial targets. Zhou et al. [20] proposed an approach that uses structural regularity to derive initial transformations, followed by 2D-3D line registration based on a nearest-neighbor technique. Calibration parameters are then estimated using correspondences from local triangle-preserving matching, and subsequently refined via the Levenberg-Marquardt algorithm. Meanwhile, an outlier removal strategy is integrated to further improve accuracy. For systems with small FoV LiDARs, Liu et al. [21] formulated multi-LiDAR extrinsic calibration as a bundle adjustment problem. They introduced an adaptive voxelization technique for efficient feature point extraction and improved the precision and convergence speed of the nonlinear optimization by deriving a second-order cost function. For cross-modal calibration across stereo, thermal, and LiDAR sensors, Fu et al. [22] minimized 3D registration errors for the stereo-LiDAR pair, while the thermal camera's extrinsics were optimized through edge feature alignment. Later, Ou et al. [23] presented a novel target-free LiDAR-camera calibration method based on cross-modality structural consistency. This method ensures global convergence within a wide search range and achieves highly accurate translational calibration even under challenging conditions. Additionally, Koide et al. [24] contributed an open-source toolbox that supports various sensor models, requiring only a single pair of LiDAR-camera data and operating autonomously without targets. Recently, methods based on Gaussian splatting have also been increasingly applied to solve extrinsic calibration problems. For instance, Zhou et al. [25] employed 2D Gaussian splatting (2DGS) for extrinsic calibration of LiDAR-camera systems. Their method reconstructs a colorless 2DGS from LiDAR point clouds and jointly updates splat colors and extrinsic parameters by minimizing a photometric loss, augmented with reprojection and triangulation losses for robustness, demonstrating strong performance on the KITTI dataset.

Despite these advances, most existing target-free calibration methods still require multiple frames of images and LiDAR point cloud data. Furthermore, most feature-based methods continue to face notable limitations in extracting point cloud and image features—limitations that severely compromise their final calibration performance. To address these challenges, we propose a novel automatic target-free LiDAR-camera calibration method, built on robust geometric edge feature extraction and visibility check. Our approach is designed to effectively mitigate the shortcomings in current feature extraction and optimization workflows.

III. Methodology

In this section, the proposed calibration method is elaborated in detail. First, its overall framework is introduced, followed by a detailed exposition of the proposed scheme for extracting geometric edge features from point clouds and images. Subsequently, the point cloud visibility check module is described in detail, and finally, the strategies for feature matching and optimization are given.

A. Problem Formulation and Overview

The extrinsic calibration of a LiDAR-camera system involves estimating the rigid transformation between the LiDAR ($\mathcal{L}$) and camera ($\mathcal{C}$) coordinate frames. This transformation is defined by a rotation matrix $\boldsymbol{R}_{\mathcal{L}}^{\mathcal{C}} \in \mathrm{SO}(3)$ and a translation vector $\boldsymbol{t}_{\mathcal{L}}^{\mathcal{C}} \in \mathbb{R}^3$, which project a point $\boldsymbol{p}_i^{\mathcal{L}}$ from $\mathcal{L}$ to its corresponding point $\boldsymbol{p}_i^{\mathcal{C}}$ in $\mathcal{C}$:

$$\boldsymbol{p}_i^{\mathcal{C}} = \boldsymbol{R}_{\mathcal{L}}^{\mathcal{C}} \boldsymbol{p}_i^{\mathcal{L}} + \boldsymbol{t}_{\mathcal{L}}^{\mathcal{C}} \tag{1}$$

To accomplish this calibration task, this paper presents an automatic target-free extrinsic calibration algorithm for LiDAR-camera systems, with an overview provided in Fig. 2. As a feature-based calibration approach, the extraction of multi-modal geometric edge features from LiDAR point clouds and the image is its core task. It should be noted that the aforementioned **geometric edge features are derived exclusively from structural information, independent of texture or other non-geometric data**. Particularly in image feature extraction, unlike traditional methods that rely on pixel intensity gradients, our proposed image geometric edge

Authorized licensed use limited to: TONGJI UNIVERSITY. Downloaded on October 18,2026 at 00:47:47 UTC from IEEE Xplore. Restrictions apply.

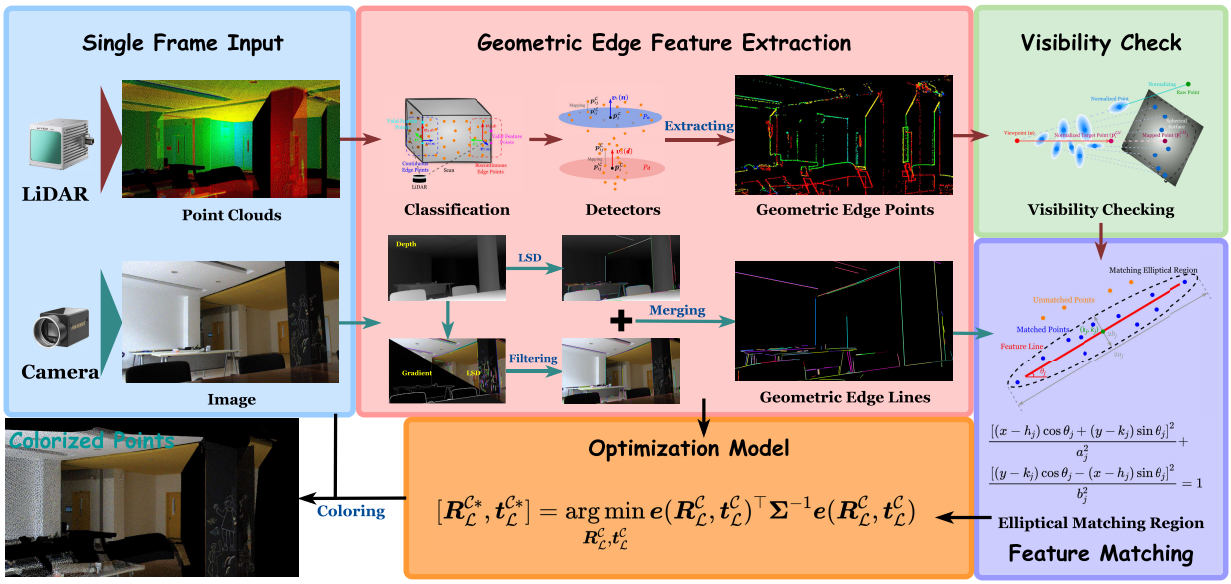


Fig. 2. The overview of the method. After acquiring the single-frame LiDAR point cloud and image data, we perform feature extraction on both modalities independently. For point clouds, we propose feature detectors to extract geometric edge points based on decomposition of local point clouds. For image, traditional line extraction method and learning-based depth estimation are combined to extract geometric edge line features. Subsequently, the visibility check module removes invisible point features, while an elliptical matching strategy then establishes correspondences between these multi-modal features. Finally, the extrinsic parameters are optimized by minimizing the re-projection error of the matched features.

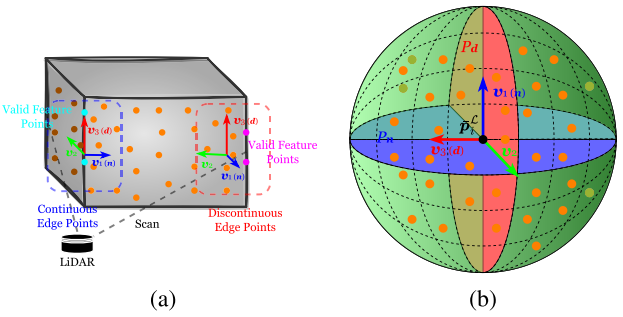


Fig. 3. Geometric edge point classification and local point cloud decomposition for LiDAR feature extraction. (a) shows the classification of LiDAR edge points based on local geometric characteristics, including continuous edge points (blue, with smooth depth variations) and discontinuous edge points (red, with abrupt depth changes). (b) shows the decomposition of local point clouds, eigenvalue decomposition is performed on the local covariance matrix to obtain eigenvectors $\boldsymbol{v}_1$ (normal vector $\boldsymbol{n}$ of the fitting plane) and $\boldsymbol{v}_3$ (primary directional vector $\boldsymbol{d}$ of point distribution). Meanwhile, their corresponding vertical planes $P_{\boldsymbol{n}}$ and $P_{\boldsymbol{d}}$ are also constructed.

features leverage depth estimation results to concentrate solely on object edges with significant depth variations.

The specific extraction procedures for geometric edge features from both point clouds and images are as follows. As illustrated in Fig. 3, for the point cloud feature extraction, our method focuses on structured point clouds and begins by decomposing local point clusters to estimate their normal vectors and primary directional vectors. Subsequently, vertical planes corresponding to these vectors are constructed, respectively, and the local points are projected onto these planes. Valid edge points exhibit distinct distribution patterns within these projections, which are leveraged to identify geometric edge features. The detailed implementation of this process is provided in Section III-B.1. For image feature extraction, traditional methods are sensitive to texture variations and lack the capability to directly yield geometric features compatible with the above-mentioned 3D edge points. To address these issues, we propose a geometric feature extraction approach that integrates learning-based depth estimation with traditional image line extraction method. This combination reduces sensitivity to texture and enables the robust extraction of geometric edge lines from the image, thereby facilitating accurate matching with LiDAR-derived edge points, with further implementation details available in Section III-B.2.

Moreover, to mitigate the adverse effect of invalid occluded features on calibration accuracy, a point cloud visibility check module is incorporated into the optimization process. Specifically, a real-time filtering strategy is introduced to remove occluded points based on iterative optimization results, preventing such invalid features from degrading the solution. Such a visibility check strategy is detailed in Section III-C.

Finally, we establish correspondences between the extracted cross-modal features and formulate an optimization model to compute the high-precision extrinsic parameters.

B. Geometric Edge Feature Extraction

As a feature-based approach, a central task involves extracting and matching corresponding features from the point clouds and the image. In this subsection, the proposed two novel methods, which are respectively used to extract geometric edge features from LiDAR point clouds and the image, are presented in detail.

1) Geometric Edge Point Extraction From Point Clouds: Point clouds acquired by LiDAR often contain significant noise, which poses a severe challenge to point cloud feature extraction. Importantly, we observe that points located at object edges exhibit stronger anti-interference capabilities

Authorized licensed use limited to: TONGJI UNIVERSITY. Downloaded on October 18,2026 at 00:47:47 UTC from IEEE Xplore. Restrictions apply.

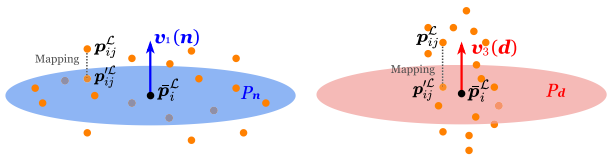


Fig. 4. Planar mapping process of point clouds. Based on the normal vector $\boldsymbol{n}$ and the directional vector $\boldsymbol{d}$ obtained from the eigenvalue decomposition of the local point clouds $\{\boldsymbol{p}_{ij}^{\mathcal{L}}\}_{j=1}^{n}$, two planes $P_{\boldsymbol{n}}$ and $P_{\boldsymbol{d}}$ are constructed, which pass through the centroid $\bar{\boldsymbol{p}}_i^{\mathcal{L}}$ and are perpendicular to $\boldsymbol{n}$ and $\boldsymbol{d}$, respectively. The planar mapping points $\boldsymbol{p}_{ij}^{\prime\mathcal{L}}$ for $\boldsymbol{p}_{ij}^{\mathcal{L}}$ are then computed using Eq. (9).

against such noise. While matrix decomposition techniques, such as singular value decomposition (SVD) and eigenvalue decomposition (EVD) [26], [27], have proven highly effective in traditional image denoising and digital image processing, adapting these methods to robustly extract these geometric edges from sparse LiDAR data remains a critical yet challenging objective. To address this gap, we propose a novel feature extraction method that leverages EVD to analyze the local spatial distribution of points around a target point $\boldsymbol{p}_i^{\mathcal{L}}$, effectively determining whether $\boldsymbol{p}_i^{\mathcal{L}}$ is a valid geometric edge point. Specifically, the local structure of a point cloud can be broadly categorized as either well-structured or unordered. Among these, we focus on extracting geometric edge point features from the structured regions, which are further classified into two types: continuous edge points and discontinuous edge points, as shown in Fig. 3(a). To quantify the local geometry, we first use a K-nearest neighbor search to select n points $\{\boldsymbol{p}_{ij}^{\mathcal{L}}\}_{j=1}^{n}$ surrounding the target point $\boldsymbol{p}_i^{\mathcal{L}}$. The structural characteristics of $\{\boldsymbol{p}_{ij}^{\mathcal{L}}\}_{j=1}^{n}$ are largely embedded in their covariance matrix $\boldsymbol{C}$:

$$\boldsymbol{C} = \frac{1}{n-1}\sum_{j=1}^{n}(\boldsymbol{p}_{ij}^{\mathcal{L}} - \bar{\boldsymbol{p}}_i^{\mathcal{L}})(\boldsymbol{p}_{ij}^{\mathcal{L}} - \bar{\boldsymbol{p}}_i^{\mathcal{L}})^{\top} \tag{2}$$

where $\bar{\boldsymbol{p}}_i^{\mathcal{L}}$ is the centroid of $\{\boldsymbol{p}_{ij}^{\mathcal{L}}\}_{j=1}^{n}$:

$$\bar{\boldsymbol{p}}_i^{\mathcal{L}} = \frac{1}{n}\sum_{j=1}^{n}\boldsymbol{p}_{ij}^{\mathcal{L}} \tag{3}$$

After that, we perform eigenvalue decomposition on the covariance matrix $\boldsymbol{C}$ to obtain eigenvalues ($\lambda_1 < \lambda_2 < \lambda_3$) and their corresponding eigenvectors ($\boldsymbol{v}_1, \boldsymbol{v}_2, \boldsymbol{v}_3$). As illustrated in Fig. 3(b), these eigenvectors have direct geometric interpretations: $\boldsymbol{v}_1$ represents the normal vector $\boldsymbol{n}$ of the local point cloud fitting plane, while $\boldsymbol{v}_3$ corresponds the primary directional vector $\boldsymbol{d}$ of the point distribution. Leveraging these geometric vectors, we construct two planes $P_{\boldsymbol{n}}$ and $P_{\boldsymbol{d}}$ (as shown in Fig. 4), which pass through the centroid $\bar{\boldsymbol{p}}_i^{\mathcal{L}}\,(\bar{x}, \bar{y}, \bar{z})^{\top}$ and are perpendicular to $\boldsymbol{n}\,(n_x, n_y, n_z)^{\top}$ and $\boldsymbol{d}\,(d_x, d_y, d_z)^{\top}$, respectively:

$$\left\{\boldsymbol{x}(x,y,z)^{\top} \in P_{\boldsymbol{n}} : |\boldsymbol{n}\cdot(\boldsymbol{x} - \bar{\boldsymbol{p}}_i^{\mathcal{L}}) = 0\right\} \tag{4}$$

$$\left\{\boldsymbol{x}(x,y,z)^{\top} \in P_{\boldsymbol{d}} : |\boldsymbol{d}\cdot(\boldsymbol{x} - \bar{\boldsymbol{p}}_i^{\mathcal{L}}) = 0\right\} \tag{5}$$

Then, each point in $\{\boldsymbol{p}_{ij}^{\mathcal{L}}\}_{j=1}^{n}$ is projected onto the planes $P_{\boldsymbol{n}}$ and $P_{\boldsymbol{d}}$, respectively. Taking plane $P_{\boldsymbol{n}}$ as an example, the detailed process of mapping the point $\boldsymbol{p}_{ij}^{\mathcal{L}}$ to its projected point $\boldsymbol{p}_{ij}^{\prime\mathcal{L}}$ is presented below. Specifically, a projection line is constructed through point $\boldsymbol{p}_{ij}^{\mathcal{L}}$ and parallel to the normal vector $\boldsymbol{n}$:

$$\boldsymbol{x}(t) = \boldsymbol{p}_{ij}^{\mathcal{L}} + \boldsymbol{n}t \tag{6}$$

where t is a scalar parameter: the point at t=0 corresponds to $\boldsymbol{p}_{ij}^{\mathcal{L}}$, and the projection point $\boldsymbol{p}_{ij}^{\prime\mathcal{L}}$ on the plane is found at t=t_0. To determine the value of t_0, the projection line equation Eq. (6) is substituted into the plane equation Eq. (4).

$$\boldsymbol{n}\cdot\left(\boldsymbol{x}(t_0)) - \bar{\boldsymbol{p}}_i^{\mathcal{L}}\right) = 0 \Rightarrow \boldsymbol{n}\cdot\boldsymbol{p}_{ij}^{\mathcal{L}} + \boldsymbol{n}\cdot\boldsymbol{n}t_0 - \boldsymbol{n}\cdot\bar{\boldsymbol{p}}_i^{\mathcal{L}} = 0 \tag{7}$$

Since $\boldsymbol{n}$ is a unit vector ($\|\boldsymbol{n}\|^2$=1), t_0 is solved as:

$$t_0 = \boldsymbol{n}\cdot(\bar{\boldsymbol{p}}_i^{\mathcal{L}} - \boldsymbol{p}_{ij}^{\mathcal{L}}) \tag{8}$$

Finally, the projected point $\boldsymbol{p}_{ij}^{\prime\mathcal{L}}$ on plane $P_{\boldsymbol{n}}$ is:

$$\boldsymbol{p}_{ij}^{\prime\mathcal{L}} = \boldsymbol{p}_{ij}^{\mathcal{L}} + \boldsymbol{n}t_0 \tag{9}$$

The projection of $\boldsymbol{p}_{ij}^{\mathcal{L}}$ onto $P_{\boldsymbol{d}}$ can be performed analogously by substituting $\boldsymbol{d}$ for $\boldsymbol{n}$ in Eqs. (6)~ (9).

To quantify the distribution of above projected points, the planes $P_{\boldsymbol{n}}$ and $P_{\boldsymbol{d}}$ are divided into uniform angular intervals, and the number of points within each interval is counted. As illustrated in Figs. 5(a)~5(d), the distribution of the projections of valid geometric edge points (including both continuous and discontinuous edge points) onto their corresponding planes $P_{\boldsymbol{n}}$ and $P_{\boldsymbol{d}}$ exhibits a distinct inhomogeneity. In contrast, the projection distribution of invalid and unordered points is nearly uniform, as shown in Fig. 5(e). To formalize this distinction, a criterion based on the proportion of continuous blank regions is introduced.

For each target point $\boldsymbol{p}_i^{\mathcal{L}}$, we calculate $\gamma_{\boldsymbol{n}}^{\max}$ and $\gamma_{\boldsymbol{d}}^{\max}$, representing the largest proportions of continuous blank angular intervals on planes $P_{\boldsymbol{n}}$ and $P_{\boldsymbol{d}}$, respectively. Finally, building on these metrics, the target point $\boldsymbol{p}_i^{\mathcal{L}}$ is deemed as a valid geometric edge point if it satisfies both of the following two conditions:

$$\gamma_{\boldsymbol{n}}^{\max} > \kappa\cdot\exp\left(\frac{\lambda_1}{\lambda_1+\lambda_2+\lambda_3}\right) \tag{10}$$

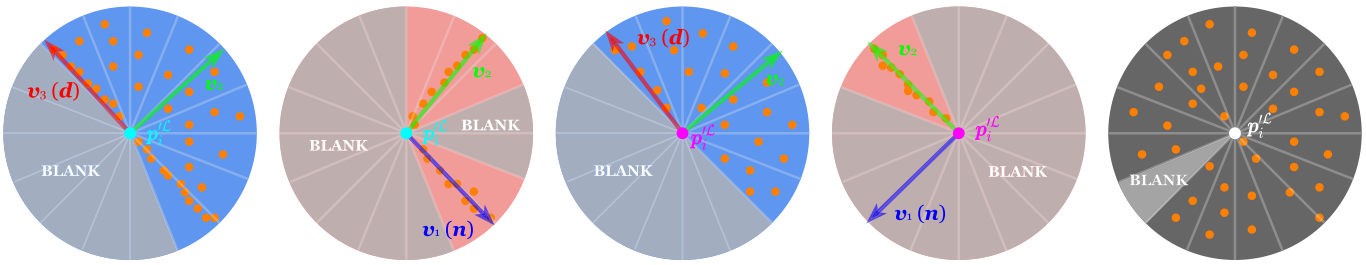
$$\gamma_{\boldsymbol{d}}^{\max} > \kappa\cdot\exp\left(\frac{1-\lambda_3}{\lambda_1+\lambda_2+\lambda_3}\right) \tag{11}$$

As a parameter, κ is set to 0.2 in this work. To validate the effectiveness of the proposed feature extraction method, Figs. 6(a) and 6(b) respectively present the raw point cloud and the corresponding edge points extracted through the aforementioned workflow.

2) Geometric Edge Line Extraction From Image: Accurate matching with the above-extracted 3D geometric edge points requires corresponding geometric edge features from the image. However, traditional 2D image feature extraction methods struggle to recover such geometric information.

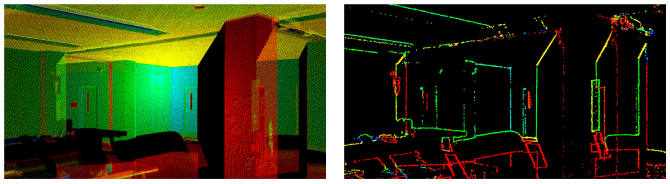
To address this issue, we propose a method that integrates learning-based depth estimation to bridge the modality gap between the 2D images and the 3D geometric edge points. Given that geometric structures often manifest as lines, we focus on extracting line features, with our pipeline implemented through the following sequential steps: First, we estimate a depth image $\boldsymbol{D}$ from the raw input image $\boldsymbol{I}$ using Depth Anything V2 [28], as shown in Fig. 7(a). Subsequently,

Authorized licensed use limited to: TONGJI UNIVERSITY. Downloaded on October 18,2026 at 00:47:47 UTC from IEEE Xplore. Restrictions apply.



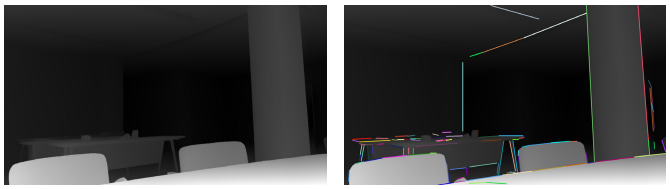
(a) Continuous edge points mapped on P_n (b) Continuous edge points mapped on P_d (c) Discontinuous edge points mapped on P_n (d) Discontinuous edge points mapped on P_d (e) Unordered points mapped on $P_{n/d}$

Fig. 5. The distribution of planar mapped point clouds distinguishes valid edge points from invalid ones. (a)~(d) illustrate the mapping results of both continuous and discontinuous edge point clouds onto their respective planes P_n and P_d. It is noteworthy that for valid edge points, their distributions are markedly inhomogeneous, featuring large, continuous blank regions. Conversely, the mapping of invalid edge points in (e) results in a uniform distribution with minimal blank areas.



(a) The raw point clouds (b) The extracted geometric edge points

Fig. 6. The results of geometric edge feature points extracted using our method. The proposed feature extraction method successfully yields a sufficient quantity of high-quality geometric edge points from the scene.

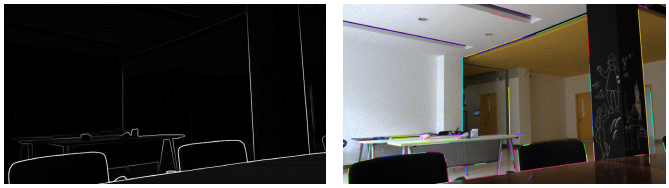


(a) (b)

Fig. 7. The geometric line features extraction from estimated depth image. We extract geometric line features from the raw image by first generating a depth image using Depth Anything V2 [28], as shown in (a). Prominent line features are then extracted from this depth image using the LSD algorithm, as illustrated in (b). However, the number of lines obtained from this initial extraction is insufficient for subsequent optimization.

the LSD algorithm is applied directly to $\boldsymbol{D}$, yielding an initial set of geometric edge lines $\{\boldsymbol{l}_i^{\boldsymbol{D}}\}_{i=1}^n$, as shown in Fig. 7(b). However, this approach alone is insufficient, yielding too few features due to the inherent generalization limitations of the depth estimation model, which often fails to produce precise depth differences across large regions.

To obtain more geometric edge lines, we propose a depth gradient checking method for extracting geometric edge lines from image, as presented in Fig. 8. As illustrated in Fig. 8(a), we first calculate a dense depth gradient map $\boldsymbol{G}$ from $\boldsymbol{D}$, and further derive the magnitude $g(\boldsymbol{x}_i)$ and angle $\varphi(\boldsymbol{x}_i)$ of the gradient at each point $\boldsymbol{x}_i$ in $\boldsymbol{G}$. Subsequently, to obtain candidate line features, the LSD algorithm is reapplied to extract line segments $\{\boldsymbol{l}_j^{\boldsymbol{I}}\}_{j=1}^m$ from the raw image $\boldsymbol{I}$. However, these line segments contain a large number of invalid, texture-based segments that lack valid geometric significance. Therefore, a key observation guides our approach: valid geometric edges



(a) (b)

Fig. 8. Geometric line feature extraction from the raw image. We enhance geometric line feature extraction from the raw image by fusing appearance-based line detection with depth geometric information. (a) shows the depth gradient acquired from the depth image. Concurrently, line segments are extracted from the raw image using the LSD algorithm. These candidate lines are then validated via depth gradient information to filter out features lacking geometric significance. The retained geometric edge lines are shown in (b).

typically exhibit large depth gradients in the depth image. Using this insight, the validity of each LSD-derived line $\boldsymbol{l}_j^{\boldsymbol{I}}$ is validated as a geometric edge via calculation of its average depth gradient.

Specifically, the Bresenham algorithm [29] is first adopted to obtain the coordinates of all points on the line segment $\boldsymbol{l}_j^{\boldsymbol{I}}$. Since the perpendicular component of the depth gradient at valid geometric edges typically exhibits significant variations, the component of the depth gradient perpendicular to the edge is adopted as the core metric for determining the validity of such geometric edges. Accordingly, the gradient score $g_{\boldsymbol{l}_j^{\boldsymbol{I}}}$ of the line segment $\boldsymbol{l}_j^{\boldsymbol{I}}$ is defined as follows:

$$g_{\boldsymbol{l}_j^{\boldsymbol{I}}} = \frac{1}{n}\sum_{i=1}^{n} |\sin(\varphi_i - \theta_j)| \cdot g_i \tag{12}$$

where n is the number of points in $\boldsymbol{l}_j^{\boldsymbol{I}}$, g_i and φ_i are depth gradient magnitude and angle at the i-th point, respectively, and θ_j is the angle of $\boldsymbol{l}_j^{\boldsymbol{I}}$.

Furthermore, we find that longer lines typically exhibit superior robustness and are more likely to represent true geometric edges. Accordingly, the line length ℓ_j, the mean length ℓ_m of all lines in $\{\boldsymbol{l}_j^{\boldsymbol{I}}\}_{j=1}^m$, and the dimensions of raw image $\boldsymbol{I}$ (width W, height H) are incorporated into the validity criterion; when the gradient score $g_{\boldsymbol{l}_j^{\boldsymbol{I}}}$ satisfies the following condition, the line feature $\boldsymbol{l}_j^{\boldsymbol{I}}$ can be determined as a valid

Authorized licensed use limited to: TONGJI UNIVERSITY. Downloaded on October 18,2026 at 00:47:47 UTC from IEEE Xplore. Restrictions apply.

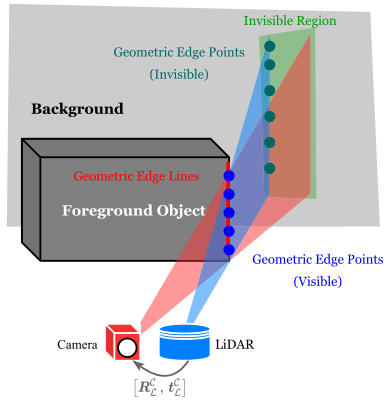


Fig. 9. Occlusion analysis of point clouds. The iterative optimization of LiDAR-camera extrinsic parameters introduces LiDAR viewpoint discrepancies. Under these changing perspectives, all visible raw point clouds originally in the background become partially invisible. These occluded points are incorrectly extracted as geometric edge points and participate in subsequent optimization, which adversely affects the accuracy of calibration results.

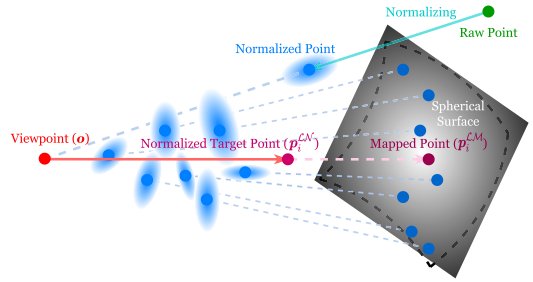


Fig. 10. Point cloud visibility check. Determining a point's visibility involves identifying whether it is occluded by other points closer to the viewpoint $\boldsymbol{o}$. Specifically, the visibility assessment of the target point is achieved by analyzing the degree of Gaussian ellipsoid overlap between the normalized target point and each normalized point in its local neighborhood, as well as the consistency of normal vectors between the two.

geometric edge:

$$g_{l_j^I} > \frac{\max(W,H)}{\ell_j \ell_m} \tag{13}$$

As illustrated in Fig. 8(b), the line features satisfying the criteria Eq. (13) are retained as valid geometric edge features $\{\boldsymbol{l}_j^{\boldsymbol{I}}\}_{j=1}^{k}$. Finally, we merge $\{\boldsymbol{l}_j^{\boldsymbol{I}}\}_{j=1}^{k}$ with the edge line $\{\boldsymbol{l}_i^{\boldsymbol{D}}\}_{i=1}^{n}$ extracted directly from the depth image to obtain a sufficient number of robust image geometric edge line features.

C. Point Cloud Visibility Check

As the extrinsic parameters are updated (Fig. 9), previously visible point cloud may become partially invisible from the camera's viewpoint due to occlusion. This causes many occluded points to be misidentified as valid geometric features. Including these points in the optimization introduces substantial errors, severely degrading calibration accuracy. Therefore, accurately identifying and removing occluded points is essential for robust calibration. Despite its importance, point cloud visibility check remains an under-explored area of research. To fill this research gap, we propose an effective visibility check method whose details are presented below.

As illustrated in Fig. 10, the raw point cloud is firstly normalized for ease of processing. For each point $\boldsymbol{p}_i^{\mathcal{L}}$ of the raw point cloud $\{\boldsymbol{p}_i^{\mathcal{L}}\}_{i=1}^{d}$, it is normalized as $\boldsymbol{p}_i^{\mathcal{LN}}$:

$$\boldsymbol{p}_i^{\mathcal{LN}} = \frac{\boldsymbol{p}_i^{\mathcal{L}}}{\text{Range}_{\max}} \tag{14}$$

where $\text{Range}_{\max}$ is the maximum measurement range of the point in the raw point cloud. For point $\boldsymbol{p}_i^{\mathcal{LN}}$, its visibility is determined by local points lying between it and the viewpoint $\boldsymbol{o}$. To acquire these local points, every $\boldsymbol{p}_i^{\mathcal{LN}}$ $(i = 1, 2, \cdots, d)$ is projected onto the unit sphere as:

$$\boldsymbol{p}_i^{\mathcal{LM}} = \left(\frac{\boldsymbol{p}_i^{\mathcal{LN}}}{\left\| \boldsymbol{p}_i^{\mathcal{LN}} \right\|} + \boldsymbol{c} \right) \tag{15}$$

where $\boldsymbol{c}$ represents the center coordinates of the sphere.

We then use a KD-tree to retrieve the m local spherical points $\{\boldsymbol{p}_{ij}^{\mathcal{LM}}\}_{j=1}^{m}$ surrounding $\boldsymbol{p}_i^{\mathcal{LM}}$ on the spherical surface. Their corresponding normalized points $\{\boldsymbol{p}_{ij}^{\mathcal{LN}}\}_{j=1}^{m}$ potentially occlude the point $\boldsymbol{p}_i^{\mathcal{L}}$, with closer points having a greater occluding effect.

To quantify this occlusion effect, we first sort $\{\boldsymbol{p}_{ij}^{\mathcal{LN}}\}_{j=1}^{m}$ by range (ascending), then calculate the occlusion contribution α_j of point $\boldsymbol{p}_{ij}^{\mathcal{LN}}$ to $\boldsymbol{p}_i^{\mathcal{L}}$ as follows. Specifically, the deviation of $\boldsymbol{p}_{ij}^{\mathcal{LN}}$ from the line of sight between $\boldsymbol{p}_i^{\mathcal{L}}$ and the viewpoint $\boldsymbol{o}$ are first considered to assess its influence on the occlusion of $\boldsymbol{p}_i^{\mathcal{L}}$. Thus, the occlusion factor τ_{1j} as the normalized perpendicular distance from $\boldsymbol{p}_{ij}^{\mathcal{LN}}$ to this line of sight is defined as:

$$\tau_{1j} = \chi_1 \frac{\left\| (\boldsymbol{p}_{ij}^{\mathcal{LN}} - \boldsymbol{o}) \times (\boldsymbol{p}_i^{\mathcal{LN}} - \boldsymbol{o}) \right\|}{\left\| \boldsymbol{p}_{ij}^{\mathcal{LN}} - \boldsymbol{o} \right\|} \in [0,1] \tag{16}$$

where χ_1 is a weight parameter.

Furthermore, the relationship between the normals of the neighborhood points also affects their mutual occlusion—points with similar normal directions exhibit a stronger occlusion effect. Therefore, we incorporate this phenomenon by defining another normal-based occlusion factor τ_{2j}:

$$\tau_{2j} = \chi_2 \frac{\boldsymbol{n}_{ij} \cdot \boldsymbol{n}_i}{\left\| \boldsymbol{n}_{ij} \right\| \left\| \boldsymbol{n}_i \right\|} \in [0,1] \tag{17}$$

where χ_2 is a weight parameter, $\boldsymbol{n}_i$ and $\boldsymbol{n}_{ij}$ are the normal vectors of points $\boldsymbol{p}_i^{\mathcal{LN}}$ and $\boldsymbol{p}_{ij}^{\mathcal{LN}}$, respectively.

Finally, the occlusion contribution α_j of $\boldsymbol{p}_{ij}^{\mathcal{LN}}$ on $\boldsymbol{p}_i^{\mathcal{L}}$ is expressed as:

$$\alpha_j = \varrho_j \exp\left(-\frac{\omega_j}{2} \left(\frac{\tau_{1j}}{\tau_{2j}} \right)^2 \right) \tag{18}$$

with ω_j being a weight parameter, and the term ϱ_j in Eq. (18) is calculated as follows:

$$\varrho_j = \begin{cases} \dfrac{\text{Range}_i - \text{Range}_{ij}}{\text{Range}_{\max}} \in [0,1] & \text{if } \text{Range}_i > \text{Range}_{ij}, \\ 0 & \text{otherwise} \end{cases} \tag{19}$$

where Range_i and Range_{ij} are measurement ranges of $\boldsymbol{p}_i^{\mathcal{L}}$ and the raw point corresponding to $\boldsymbol{p}_{ij}^{\mathcal{LN}}$, respectively.

Authorized licensed use limited to: TONGJI UNIVERSITY. Downloaded on October 18,2026 at 00:47:47 UTC from IEEE Xplore. Restrictions apply.

Algorithm 1 :Point Cloud Visibility Check

Require: Raw point clouds $\{\boldsymbol{p}_i^{\mathcal{L}}\}_{i=1}^{d}$ and the current extrinsic calibration results $[\boldsymbol{R}_{\mathcal{C}}^{\mathcal{L}}, \boldsymbol{t}_{\mathcal{C}}^{\mathcal{L}}]$
Ensure: The visible points $\{\boldsymbol{p}_k^{\mathcal{V}}\}_{k=1}^{v}$
Normalize the raw point $\{\boldsymbol{p}_i^{\mathcal{L}}\}_{i=1}^{d}$ to $\{\boldsymbol{p}_i^{\mathcal{LN}}\}_{i=1}^{d}$ using Eq. (14)
Map the normalized point $\{\boldsymbol{p}_i^{\mathcal{LN}}\}_{i=1}^{d}$ to $\{\boldsymbol{p}_i^{\mathcal{LM}}\}_{i=1}^{d}$ using Eq. (15)
for each point $\boldsymbol{p}_i^{\mathcal{LN}}$ in $\{\boldsymbol{p}_i^{\mathcal{LN}}\}_{i=1}^{d}$ **do**
  Retrieve the local normalized points $\{\boldsymbol{p}_{ij}^{\mathcal{LN}}\}_{j=1}^{m}$ of $\boldsymbol{p}_i^{\mathcal{LN}}$ based on the corresponding mapped point $\boldsymbol{p}_i^{\mathcal{LM}}$
  for each point $\boldsymbol{p}_{ij}^{\mathcal{LN}}$ in $\{\boldsymbol{p}_{ij}^{\mathcal{LN}}\}_{j=1}^{m}$ **do**
    Calculate the occlusion factor ϱ_j using Eq. (19)
    Calculate the occlusion factor τ_{1j} using Eq. (16)
    Calculate the occlusion factor τ_{2j} using Eq. (17)
    Calculate the occlusion contribution $\hat{\alpha}_j$ using Eq. (20)
  end for
  Obtain the visibility score $\mathcal{V}_{\boldsymbol{p}_i^{\mathcal{L}}}$ of $\boldsymbol{p}_i^{\mathcal{L}}$ using Eq. (21)
  if $\mathcal{V}_{\boldsymbol{p}_i^{\mathcal{L}}} > \nu$ **then**
    Add $\boldsymbol{p}_i^{\mathcal{L}}$ into $\{\boldsymbol{p}_k^{\mathcal{V}}\}_{k=1}^{v}$
  end if
end for
return $\{\boldsymbol{p}_k^{\mathcal{V}}\}_{k=1}^{v}$

It should be emphasized that the occlusion effect of $\boldsymbol{p}_{ij}^{\mathcal{LN}}$ on $\boldsymbol{p}_i^{\mathcal{L}}$ is also influenced by the visibility of $\boldsymbol{p}_{ij}^{\mathcal{LN}}$ itself. Thus, to incorporate this visibility into consideration, the adjusted occlusion contribution of $\boldsymbol{p}_{ij}^{\mathcal{LN}}$ to $\boldsymbol{p}_i^{\mathcal{L}}$ is formulated as follows:

$$\hat{\alpha}_j = \alpha_j(1 - \hat{\alpha}_{j-1}) \tag{20}$$

Subsequently, we iteratively calculate the occlusion contributions for all selected points $\{\boldsymbol{p}_{ij}^{\mathcal{LN}}\}_{j=1}^{m}$ using above-mentioned process, and construct the overall visibility score of point $\boldsymbol{p}_i^{\mathcal{L}}$:

$$\mathcal{V}_{\boldsymbol{p}_i^{\mathcal{L}}} = 1 - \sum_{j=1}^{m} \hat{\alpha}_j \tag{21}$$

If the value of $\mathcal{V}_{\boldsymbol{p}_i^{\mathcal{L}}}$ exceeds the threshold ν, the point $\boldsymbol{p}_i^{\mathcal{L}}$ is considered visible. The complete procedure for this visibility check module is detailed in Algorithm 1.

D. Feature Matching and Calibration

After extracting N visible geometric edge points $\{\boldsymbol{p}_i^{\mathcal{L}}\}_{i=1}^{N}$ from the LiDAR point cloud and M geometric edge lines $\{\boldsymbol{l}_j : |\boldsymbol{l}_j = (h_j, k_j, \theta_j, \ell_j)\}_{j=1}^{M}$ from the image—where (h_j, k_j) denotes the midpoint of $\boldsymbol{l}_j$, θ_j and ℓ_j represent the angle and length of $\boldsymbol{l}_j$, respectively—the next step is to establish correspondences $\{\boldsymbol{p}_i^{\mathcal{L}}\}_{i=1}^{N_j} \rightleftharpoons \boldsymbol{l}_j$ for each $\boldsymbol{l}_j$ with N_j geometric edge points from $\{\boldsymbol{p}_i^{\mathcal{L}}\}_{i=1}^{N}$. Since image line features are generally more robust to noise than LiDAR points, we leverage the image lines as reference and project each edge point $\boldsymbol{p}_i^{\mathcal{L}}$ into the camera's normalized image plane as $\boldsymbol{p}_i^{\mathcal{I}}$:

$$\boldsymbol{p}_i^{\mathcal{I}} = \frac{1}{z_i^{\mathcal{C}}} \boldsymbol{K}(\boldsymbol{R}_{\mathcal{L}}^{\mathcal{C}} \boldsymbol{p}_i^{\mathcal{L}} + \boldsymbol{t}_{\mathcal{L}}^{\mathcal{C}}) = \frac{1}{z_i^{\mathcal{C}}} \boldsymbol{K} \boldsymbol{p}_i^{\mathcal{C}} = \left(u_i, v_i, 1\right)^{\top} \tag{22}$$

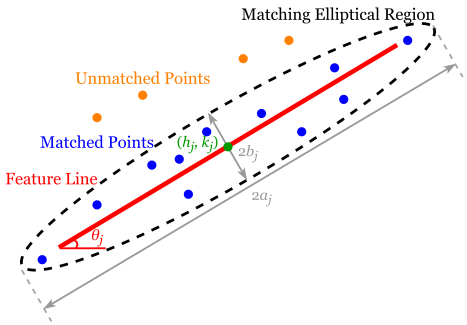


Fig. 11. Feature Matching. A matching elliptical region is constructed around each line feature $\boldsymbol{l}_j$. The ellipse is centered at the $\boldsymbol{l}_j$'s midpoint (h_j, k_j), and the two foci of the ellipse are the endpoints of $\boldsymbol{l}_j$. Thus, the semi-major axis of a_j=$0.6\ell_j$ and the semi-minor axis b_j= $\sqrt{a_j^2 - (0.5\ell_j)^2}$, where ℓ_j is the length of $\boldsymbol{l}_j$. All projected points within this ellipse are considered candidate matches for $\boldsymbol{l}_j$.

where (u_i, v_i) denote the coordinates of projected point $\boldsymbol{p}_i^{\mathcal{I}}$, $\boldsymbol{p}_i^{\mathcal{C}}$ is the point transformed into the camera frame via $[\boldsymbol{R}_{\mathcal{L}}^{\mathcal{C}}, \boldsymbol{t}_{\mathcal{L}}^{\mathcal{C}}]$ (Eq. (1)), $z_i^{\mathcal{C}}$ represents the z-axis component of $\boldsymbol{p}_i^{\mathcal{C}}$, and $\boldsymbol{K}$ is the camera intrinsic matrix:

$$\boldsymbol{K} = \begin{bmatrix} f_x & 0 & c_x \\ 0 & f_y & c_y \\ 0 & 0 & 1 \end{bmatrix} \tag{23}$$

Different from the existing methods that extract point-like image features (e.g., Canny detector) amenable to K-nearest neighbor matching, our approach instead leverages line features. Consequently, to establish correspondences between edge lines and projected points, an elliptical region is constructed around each line segment $\boldsymbol{l}_j$ (Fig. 11), defined by:

$$\frac{\left[(x - h_j)\cos\theta_j + (y - k_j)\sin\theta_j\right]^2}{a_j^2} + \frac{\left[(y - k_j)\cos\theta_j - (x - h_j)\sin\theta_j\right]^2}{b_j^2} = 1 \tag{24}$$

where a_j and b_j are the semi-major and semi-minor axes of the elliptical region, respectively, and a_j=$0.6\ell_j$ in this work. Additionally, the two foci of the elliptical region are the endpoints of $\boldsymbol{l}_j$, thus b_j= $\sqrt{a_j^2 - (0.5\ell_j)^2}$. A correspondence $\boldsymbol{p}_i^{\mathcal{L}} \rightleftharpoons \boldsymbol{l}_j$ is established when the projected point $\boldsymbol{p}_i^{\mathcal{I}}$ falls within the elliptical region of $\boldsymbol{l}_j$, and the quality of the extrinsic parameters is evaluated by the alignment between these matched features. To quantify this alignment, we minimize the distance between $\boldsymbol{p}_i^{\mathcal{I}}$ and $\boldsymbol{l}_j$, which serves as the re-projection error e_{ij}:

$$e_{ij} = \frac{A_j u_i + B_j v_i + C_j}{L_j} \tag{25}$$

where A_j, B_j, and C_j are parameters of the matched line feature $\boldsymbol{l}_j(A_j x + B_j y + C_j = 0)$, and L_j = $\sqrt{A_j^2 + B_j^2}$.

Then, we calculate the corresponding distances for each correspondence $\{\boldsymbol{p}_i^{\mathcal{L}}\}_{i=1}^{N_j} \rightleftharpoons \boldsymbol{l}_j$ using Eq. (25) and construct the

Authorized licensed use limited to: TONGJI UNIVERSITY. Downloaded on October 18,2026 at 00:47:47 UTC from IEEE Xplore. Restrictions apply.

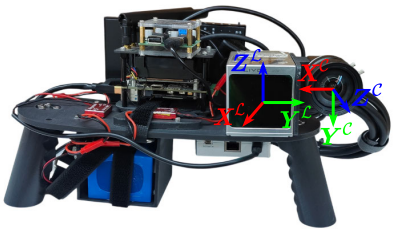
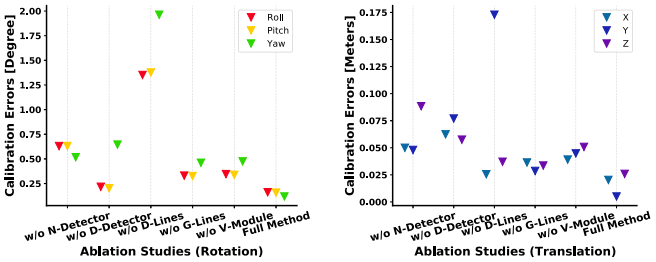


Fig. 12. The LiDAR-camera experimental setup, featuring a Livox-AVIA LiDAR and a HIKROBOT MV-CA050-12UC camera.



(a) (b)

Fig. 13. The results of the ablation study for different modules. (a) and (b) depict the errors of the rotational and translational components in the calibration results, respectively. The degradation in calibration accuracy observed when any module is removed confirms the contribution of each component to the overall system performance.

re-projection error vector $\boldsymbol{e}$ of system as follows:

$$\boldsymbol{e} = \left\{ \begin{bmatrix} e_{11} & e_{21} & \cdots & e_{N_1 1} \end{bmatrix}^\top \begin{bmatrix} e_{12} & e_{22} & \cdots & e_{N_2 2} \end{bmatrix}^\top \begin{bmatrix} \vdots & \vdots & \ddots & \vdots \end{bmatrix} \begin{bmatrix} e_{1M} & e_{2M} & \cdots & e_{N_M M} \end{bmatrix}^\top \right\} \in \mathbb{R}^H \tag{26}$$

where $H = \sum_{j=1}^{M} N_j$. Finally, we formulate the extrinsic calibration as a nonlinear least-squares problem and establish the following optimization model to minimize the re-projection errors $\boldsymbol{e}$:

$$[\boldsymbol{R}_{\mathcal{L}}^{\mathcal{C}*}, \boldsymbol{t}_{\mathcal{L}}^{\mathcal{C}*}] = \arg\min_{\boldsymbol{R}_{\mathcal{L}}^{\mathcal{C}}, \boldsymbol{t}_{\mathcal{L}}^{\mathcal{C}}} \boldsymbol{e}(\boldsymbol{R}_{\mathcal{L}}^{\mathcal{C}}, \boldsymbol{t}_{\mathcal{L}}^{\mathcal{C}})^\top \boldsymbol{\Sigma}^{-1} \boldsymbol{e}(\boldsymbol{R}_{\mathcal{L}}^{\mathcal{C}}, \boldsymbol{t}_{\mathcal{L}}^{\mathcal{C}}) \tag{27}$$

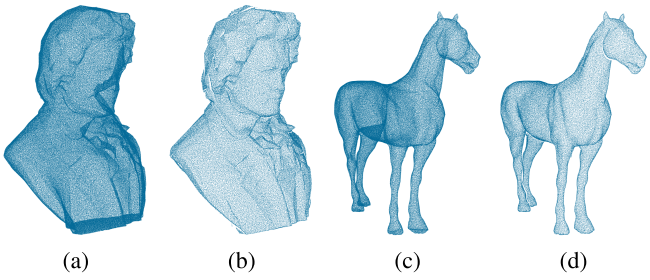
where $\boldsymbol{\Sigma}^{-1} = \frac{1}{\sigma_{ij}} \boldsymbol{I}_{H \times H}$ is the covariance matrix associated with the reliability of the correspondences. Here, σ_{ij} is defined as $\sigma_{ij} = \exp\left(\frac{d_{ij}^s + d_{ij}^e}{\ell_j} - 1\right)$, where d_{ij}^s and d_{ij}^e represent the distances from $\boldsymbol{p}_j^{\mathcal{I}}$ to the start and end points of the matched line $\boldsymbol{l}_j$, respectively.

The optimization is solved via gradient-based methods, requiring the Jacobian of the re-projection error with respect to the pose parameters. Let the six-dimensional vector $\boldsymbol{\xi} \in \mathfrak{se}(3)$ be the Lie algebra counterpart of the transformation matrix $\boldsymbol{T}_{\mathcal{L}}^{\mathcal{C}} \in SE(3)$:

$$\boldsymbol{T}_{\mathcal{L}}^{\mathcal{C}} = \begin{bmatrix} \boldsymbol{R}_{\mathcal{L}}^{\mathcal{C}} & \boldsymbol{t}_{\mathcal{L}}^{\mathcal{C}} \\ \boldsymbol{0} & 1 \end{bmatrix} \tag{28}$$

Accordingly, the Jacobian matrix of re-projection error vector $\boldsymbol{e}$ is expressed as follows:

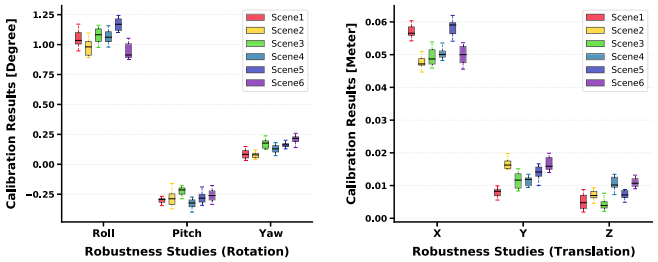
$$\boldsymbol{J} = \left\{ \begin{bmatrix} \boldsymbol{J}_{11}^\top & \boldsymbol{J}_{21}^\top & \cdots & \boldsymbol{J}_{N_1 1}^\top \end{bmatrix}^\top \begin{bmatrix} \boldsymbol{J}_{12}^\top & \boldsymbol{J}_{22}^\top & \cdots & \boldsymbol{J}_{N_2 2}^\top \end{bmatrix}^\top \begin{bmatrix} \vdots & \vdots & \ddots & \vdots \end{bmatrix} \begin{bmatrix} \boldsymbol{J}_{1M}^\top & \boldsymbol{J}_{2M}^\top & \cdots & \boldsymbol{J}_{N_M M}^\top \end{bmatrix}^\top \right\} \tag{29}$$



(a) (b) (c) (d)

(e) (f) (g)

Fig. 14. The results of point cloud visibility check module. (a), (c), and (e) display the full point clouds, which include both visible and occluded points. (b), (d), (f), and (g) show only the points visible from the current viewpoint. Notably, (f) and (g) visualize the front and back surfaces of a keyboard. Despite the close proximity of the two surfaces, the module successfully distinguishes them, demonstrating its effectiveness.



(a) (b)

Fig. 15. The results of the robustness study across different scenes. (a) and (b) depict the rotational and translational convergence performance, respectively. These findings demonstrate that the system achieves consistent convergence to a narrow range of values across various scenes and initial poses, thereby highlighting its remarkable robustness.

$$\boldsymbol{J}_{ij} = \frac{\partial e_{ij}}{\partial \boldsymbol{p}_i^{\mathcal{I}\top}} \frac{\partial \boldsymbol{p}_i^{\mathcal{I}}}{\partial \boldsymbol{p}_i^{\mathcal{C}\top}} \frac{\partial \boldsymbol{p}_i^{\mathcal{C}}}{\partial \delta \boldsymbol{\xi}^\top} = \left[-\frac{A_j f_x}{L_j z_i} \quad -\frac{B_j f_y}{L_j z_i} \quad \frac{A_j f_x x_i + B_j f_y y_i}{L_j z_i^2} \quad \frac{A_j f_x x_i y_i + B_j f_y (y_i^2 + z_i^2)}{L_j z_i^2} \quad -\frac{A_j f_x (x_i^2 + z_i^2) + B_j f_y x_i y_i}{L_j z_i^2} \quad \frac{A_j f_x y_i - B_j f_y x_i}{L_j z_i} \right] \in \mathbb{R}^6 \tag{34}$$

Authorized licensed use limited to: TONGJI UNIVERSITY. Downloaded on October 18,2026 at 00:47:47 UTC from IEEE Xplore. Restrictions apply.

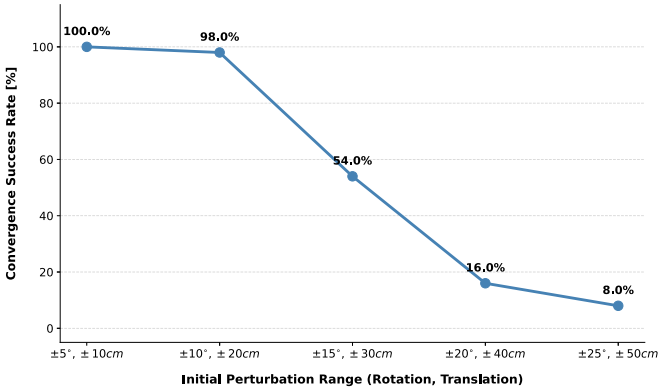
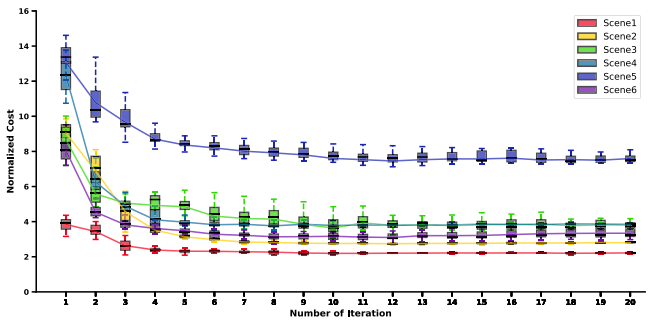
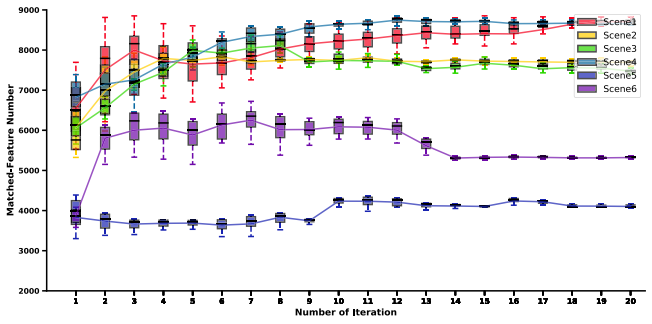


Fig. 16. Convergence success rate of the proposed method under varying scales of initial pose perturbations. The results indicate that our method maintains a highly robust success rate of 98.0% within an initial error bound of [±10°, ±20 cm]. However, as the perturbation range expands to [±15°, ±30 cm] and beyond, the success rate drops significantly, ultimately dropping to 8.0%.



(a)



(b)

Fig. 17. The results of the robustness and convergence across different scenes. (a) demonstrates the normalized cost decreases monotonically in all scenes, indicating stable convergence. Meanwhile, (b) shows that the number of successfully matched features increases steadily before plateauing in most scenes, confirming the optimization's robustness across different scenes.

where $\boldsymbol{J}_{ij}$ is the Jacobian matrix of re-projection error e_{ij} and it is computed as:

$$\boldsymbol{J}_{ij} = \frac{\partial e_{ij}}{\partial \delta \boldsymbol{\xi}^\top} = \frac{\partial e_{ij}}{\partial \boldsymbol{p}_i^{\mathcal{I}\top}} \frac{\partial \boldsymbol{p}_i^{\mathcal{I}}}{\partial \boldsymbol{p}_i^{\mathcal{C}\top}} \frac{\partial \boldsymbol{p}_i^{\mathcal{C}}}{\partial \delta \boldsymbol{\xi}^\top} \tag{30}$$

where $\delta\boldsymbol{\xi}$ is a small perturbation, and the first term in Eq. (30) is derived from Eq. (22) and Eq. (25) as:

$$\frac{\partial e_{ij}}{\partial \boldsymbol{p}_i^{\mathcal{I}\top}} = \begin{bmatrix} \frac{A_j}{L_j} & \frac{B_j}{L_j} \end{bmatrix} \tag{31}$$

TABLE I

ABLATION STUDY RESULTS ON DIFFERENT MODULES [° & meter]

Method	Roll	Pitch	Yaw	X	Y	Z
w/o **N-Detector**	0.6305	0.6308	0.5174	0.0500	0.0580	0.0884
w/o **D-Detector**	0.2171	0.2025	0.6452	0.0624	0.0770	0.0575
w/o **D-Line**	1.3512	1.3767	1.9628	0.0256	0.1729	0.0371
w/o **G-Line**	0.3304	0.3262	0.4609	0.0364	0.0286	0.0336
w/o **V-Module**	0.3458	0.3386	0.4747	0.0391	0.0448	0.0508
Full Method (Ours)	**0.1619**	**0.1579**	**0.1186**	**0.0203**	**0.0050**	**0.0259**

The second term of Eq. (30) is derived from the camera projection model by:

$$\frac{\partial \boldsymbol{p}_i^{\mathcal{I}}}{\partial \boldsymbol{p}_i^{\mathcal{C}\top}} = -\begin{bmatrix} \frac{\partial u_i}{\partial x_i} & \frac{\partial u_i}{\partial y_i} & \frac{\partial u_i}{\partial z_i} \\ \frac{\partial v_i}{\partial x_i} & \frac{\partial v_i}{\partial y_i} & \frac{\partial v_i}{\partial z_i} \end{bmatrix} = -\begin{bmatrix} \frac{f_x}{z_i} & 0 & -\frac{f_x x_i}{z_i^2} \\ 0 & \frac{f_y}{z_i} & -\frac{f_y y_i}{z_i^2} \end{bmatrix} \tag{32}$$

The third term of Eq. (30) comes from the Lie algebra perturbation model:

$$\frac{\partial \boldsymbol{p}_i^{\mathcal{C}}}{\partial \delta \boldsymbol{\xi}^\top} = \frac{\partial \boldsymbol{T}_{\mathcal{C}\mathcal{L}}^{\mathcal{C}} \boldsymbol{p}_i^{\mathcal{L}}}{\partial \delta \boldsymbol{\xi}^\top} \Rightarrow \begin{bmatrix} \boldsymbol{I} & -\boldsymbol{p}_i^{\mathcal{L}\wedge} \end{bmatrix}_{3\times 6} \tag{33}$$

where $(\cdot)^\wedge$ denotes the skew-symmetric operator. Consequently, the complete form of $\boldsymbol{J}_{ij}$ is presented in Eq. (34), as shown at the bottom of the previous page.

IV. EXPERIMENTAL RESULTS AND ANALYSIS

A. Datasets and Implementation

To validate the performance of our proposed method, we collected data from multiple diverse indoor and outdoor scenes (Fig. 18, first row) using the experimental setup in Fig. 12—comprising a Livox-AVIA LiDAR and HIKROBOT MV-CA050-12UC camera with fixed inter-sensor relative pose. To establish the ground truth for quantitative evaluation, we adopt the original CAD data of the equipment and integrate the relative pose information of core components within the LiDAR and camera. Through the aforementioned steps, the extrinsic parameter ground truth is jointly constructed, which serves as the benchmark for evaluating calibration performance.

For dense point cloud acquisition, each scene was statically recorded for approximately 10 seconds. Concurrently, all experiments were conducted on a laptop with an Intel Core i7-10510U CPU (1.80GHz×8), 32 GB RAM, without GPU acceleration. Furthermore, for comparative analysis, our method was evaluated against four SOTA target-free calibration schemes: Pandey et al. [30], Yuan et al. [31], Koide et al. [24], and MFCalib [1].

B. Ablation Studies

We perform ablation studies to demonstrate how the core modules in our method affect the calibration results by comparing our **Full Method** with five baselines without (w/o) core modules. These core modules are 1) **N-Detector**: The normal detector in the edge point extraction; 2) **D-Detector**: The directional detector in the edge point extraction; 3) **D-Line**: The edge lines extracted directly from the depth image; 4) **G-Line**: The gradient-filtered edge lines;

Authorized licensed use limited to: TONGJI UNIVERSITY. Downloaded on October 18,2026 at 00:47:47 UTC from IEEE Xplore. Restrictions apply.

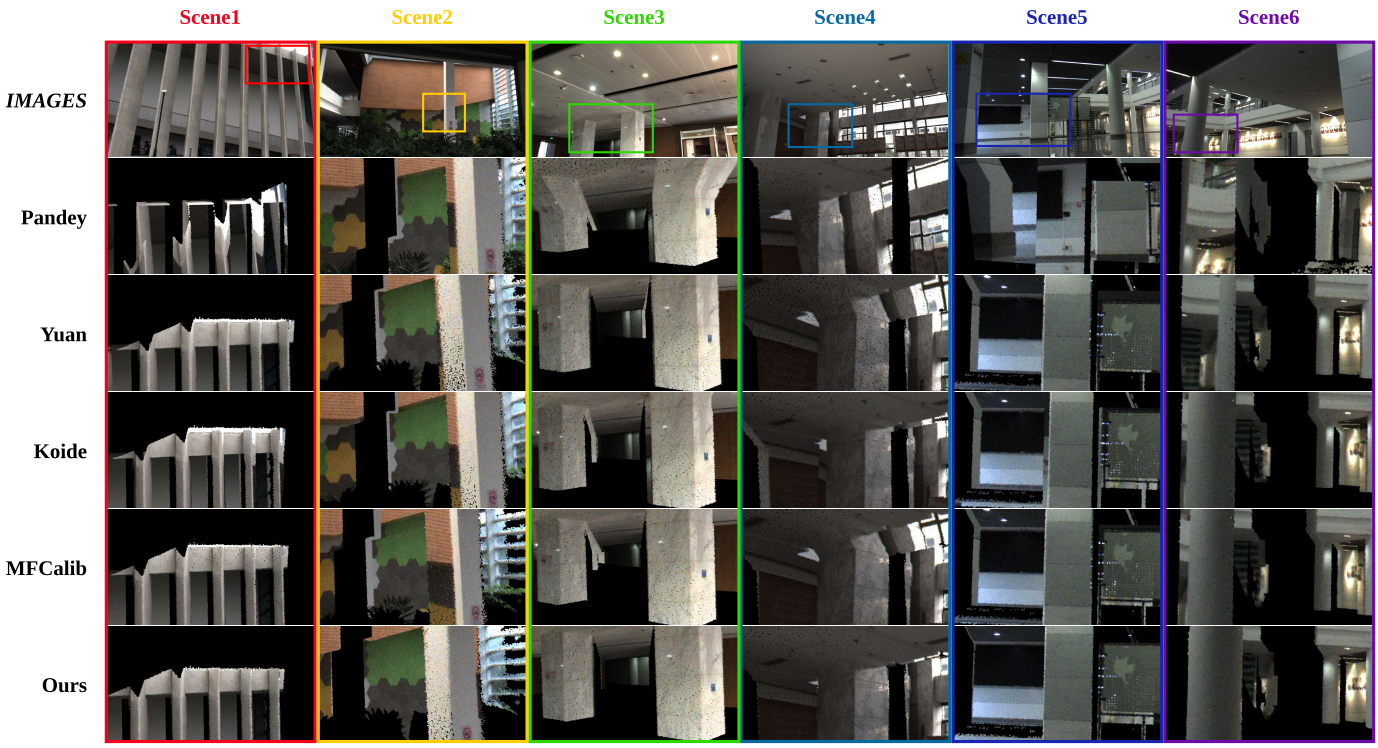


Fig. 18. Qualitative comparison of our method with SOTA approaches (Pandey et al. [30], Yuan et al. [31], Koide et al. [24], and MFCalib [1]) across various scenes. Each column represents a distinct scene, with the top row showing reference images and marked regions of interest (ROIs). Compared to other methods that exhibit noticeable projection misalignments and "color bleeding", our approach ensures pixel-level alignment between point clouds and geometric edges. This effectively maintains distinct, consistent boundaries between foreground and background objects, particularly in the ROI areas.

5) **V-Module**: The point cloud visibility check module. The results, presented in Fig. 13 and Table I, confirm that each module is essential for optimal performance.

Among these core modules, the **N-Detector** and **D-Detector**—two eigenvector-based detectors employed for point cloud feature extraction (as shown in Fig. 4)—exhibit distinct impacts on calibration accuracy. Specifically, the ablation study reveals that **N-Detector** has a greater influence on improving the estimation accuracy of rotation parameters in the extrinsic matrix, while **D-Detector** is more influential for translation part. This can be attributed to their respective sensitivities to different geometric structures. The **N-Detector** excels in planar regions, where normal vectors are well-defined, enabling robust extraction of more edge points on planes that are highly sensitive to rotational misalignment. Conversely, **D-Detector** is more effective on elongated structures (e.g., poles), where directional vectors are prominent. It reliably captures the edge points of such structures, which are more sensitive to translational errors. This makes **D-Detector** more critical for improving the estimation accuracy of translation parameters in the extrinsic matrix.

In terms of image feature extraction, to evaluate the contribution of **D-Line** and **G-Line** modules to the method's performance, we calculated the average improvement of our **Full Method** over two ablated versions—"w/o **D-Line**" and "w/o **G-Line**"—across all metrics. The experimental results presented in Table I show that **D-Line** and **G-Line** improve calibration accuracy by 69.8% and 54.4%, respectively. As we can see, **D-Line** plays a more substantial role in the calibration process compared to **G-Line**. This highlights the significant potential of depth-based methods for extracting robust geometric features.

As one of the key innovations in this work, **V-Module** has also been proven to have a high impact on the calibration results. Fig. 14 visually demonstrates how this module effectively filters out invisible point clouds from the current viewpoint, which demonstrates its effectiveness in eliminating invisible feature points during the extrinsic optimization process. Additionally, quantitative results in Table I show that, compared with the method "w/o **V-Module**", **Full Method** achieves an average accuracy improvement of 60.5% with respect to the rotation metrics (Roll, Pitch, and Yaw) and 62.0% with respect to the translation metrics (X, Y, and Z), respectively.

In summary, the ablation results demonstrate that each proposed module makes a significant and distinct contribution to the overall robustness and accuracy of the calibration system.

C. Robustness and Convergence Validation

To evaluate the robustness of our method across different scenes and its adaptability to varying initial sensor poses, 10 independent experiments were conducted per scene with random initial poses (within $\pm 5^\circ$ in rotation and ± 10 cm in translation relative to the ground truth). The corresponding results are shown in Fig. 15 and Table II. As described in Table II, the calibration results demonstrate that across

Authorized licensed use limited to: TONGJI UNIVERSITY. Downloaded on October 18,2026 at 00:47:47 UTC from IEEE Xplore. Restrictions apply.

TABLE II
ROBUSTNESS EVALUATION OF OUR METHOD ACROSS DIFFERENT SCENES [° & meter]

Scenes	Roll			Pitch			Yaw			X			Y			Z		
	Med.	Mean	Std.	Med.	Mean	Std.	Med.	Mean	Std.	Med.	Mean	Std.	Med.	Mean	Std.	Med.	Mean	Std.
Scene 1	1.0219	1.0479	0.0796	-0.2905	-0.2992	0.0218	0.0789	0.0831	0.0408	0.0566	0.0564	0.0019	0.0083	0.0079	0.0015	0.0048	0.0049	0.0026
Scene 2	0.9905	0.9710	0.0796	-0.2876	-0.2809	0.0667	0.0789	0.0751	0.0273	0.0472	0.0477	0.0021	0.0159	0.0166	0.0016	0.0070	0.0071	0.0015
Scene 3	1.0821	1.0875	0.0696	-0.2138	-0.2228	0.0361	0.1772	0.1695	0.0369	0.0487	0.0483	0.0025	0.0113	0.0114	0.0024	0.0042	0.0042	0.0015
Scene 4	1.0590	1.0693	0.0672	-0.3127	-0.3263	0.0376	0.1293	0.1285	0.0350	0.0499	0.0503	0.0017	0.0122	0.0115	0.0014	0.0099	0.0101	0.0020
Scene 5	1.1694	1.1651	0.0526	-0.2823	-0.2870	0.0460	0.1623	0.1634	0.0247	0.0589	0.0583	0.0025	0.0142	0.0141	0.0021	0.0064	0.0072	0.0015
Scene 6	0.9127	0.9341	0.0696	-0.2590	-0.2580	0.0491	0.2172	0.2072	0.0352	0.0505	0.0501	0.0030	0.0158	0.0166	0.0022	0.0107	0.0109	0.0015

various scenes and initial poses, the calibration outcomes of our method remain consistently clustered, with rotation errors varying by less than 0.25° and translation errors within 1.1 cm. This consistency demonstrates our method's strong robustness and compatibility with various device configurations and environmental conditions.

To comprehensively evaluate the convergence performance of our method under substantial initial pose deviations, we expanded the scope of our robustness tests. Extending beyond the baseline error bounds of [±5°, ±10 cm], we evaluated four progressively wider perturbation intervals: [±10°, ±20 cm], [±15°, ±30 cm], [±20°, ±40 cm], and [±25°, ±50 cm]. The corresponding convergence success rates are illustrated in Fig. 16.

As shown in Fig. 16, our method maintains a robust convergence success rate of 98.0% within the [±10°, ±20 cm] interval, demonstrating that the nonlinear optimization model effectively avoids local optima under standard engineering tolerances. However, as the perturbation range expands to [±15°, ±30 cm] and beyond, the success rate experiences a sharp inflection point, declining precipitously to 54.0%, and ultimately deteriorating to 16.0% and 8.0% at the most extreme intervals. This performance degradation stems from the invalidation of the elliptical matching constraints under severe initial errors, compounded by data association ambiguities in highly structured environments. Inherently, the elliptical matching region is designed to enforce high-precision local geometric constraints. Under excessively large rotational or translational perturbations, 3D LiDAR edge points undergo severe spatial displacement when projected onto the 2D image. This substantial shift causes the majority of projected points to fall completely outside the effective capture range of their true corresponding edge lines, thereby depriving the optimization process of valid geometric constraints. This precision-driven framework ensures exceptional alignment within a reliable capture range of [±10°, ±20 cm], providing a robust operational margin that effectively accommodates mechanical tolerances and coarse initialization errors in practical deployments.

Furthermore, to assess the rapid convergence of our method in different scenes, Figs. 17(a) and 17(b) illustrate the normalized cost $\frac{1}{N_{\text{match}}}\sum_{i=1}^{N_{\text{match}}} \boldsymbol{e}^\top \boldsymbol{\Sigma}^{-1}\boldsymbol{e}$ and the number of matched edge features N_{match} during the iterative optimization process, respectively. The results show that the normalized cost converges to a minimal value in all scenes, while the number of matched features increases steadily before stabilizing at a high level in most scenes. Notably, a performance discrepancy emerges in Scene 5: both the normalized cost and the number of matched features underperformed compared to other scenes, which can be primarily attributed to the scene's inherent structural complexity. This scene involves significant distance variations and ground reflection artifacts, which pose substantial challenges to the calibration process. Despite these difficulties, our method still achieves competitive performance under such challenging conditions compared to other approaches, as discussed in the following subsection.

D. Comparison Experiments

1) Qualitative Analysis: A qualitative comparison of our method against SOTA approaches (Pandey et al. [30], Yuan et al. [31], Koide et al. [24], and MFCalib [1]) across diverse scenes is presented in Fig. 18. The evaluation covers indoor and outdoor environments with large distance variations and challenging conditions like ground reflections, effectively testing each method's adaptability. Among them, Scene 1 is an outdoor environment that poses significant challenges to the feature extraction strategies employed by different methods. As the region highlighted by the colored rectangles in the scene image, our method achieves more accurate calibration, which is visually confirmed by precise foreground and background coloring. Furthermore, Scenes 2, 3, and 4 contain numerous typical pillar-like structures in the foreground. Conventional methods lacking point cloud visibility check incorporate a large number of invalid invisible points into the optimization process. In contrast, our visibility check module effectively filters out these outliers, resulting in superior alignment, particularly for foreground objects. Finally, Scenes 5 and 6, characterized by drastic depth changes and severe ground reflectance, test algorithmic robustness. Although these challenging conditions stress feature extraction, matching, and outlier rejection, our method effectively handles these challenges and delivers superior performance by leveraging the proposed geometric edge feature extraction framework for both point clouds and images, combined with the point cloud visibility check module.

2) Quantitative Analysis: Table III presents the quantitative performance of different calibration methods across various scenes, as well as the average performance of each method across all scenes. To ensure a rigorous comparative analysis, these calibration results of each method are evaluated separately in terms of the rotation error $E_{\boldsymbol{R}}$, translation error $E_{\boldsymbol{t}}$, and the pixel-level reprojection error E_{pixel} defined as:

$$E_{\boldsymbol{R}} = \arccos\left(\frac{\text{tr}(\boldsymbol{R}_{\text{true}}^\top \boldsymbol{R}^*) - 1}{2}\right)$$

$$E_{\boldsymbol{t}} = \|\boldsymbol{t}_{\text{true}} - \boldsymbol{t}^*\|$$

Authorized licensed use limited to: TONGJI UNIVERSITY. Downloaded on October 18,2026 at 00:47:47 UTC from IEEE Xplore. Restrictions apply.

TABLE III
COMPARATIVE ANALYSIS OF RESULTS FROM DIFFERENT METHODS ON DIFFERENT SCENES [° , meter & pixel]

Methods	Scene1			Scene2			Scene3			Scene4			Scene5			Scene6			Average		
	E_R	E_t	E_{pixel}	E_R	E_t	E_{pixel}	E_R	E_t	E_{pixel}	E_R	E_t	E_{pixel}	E_R	E_t	E_{pixel}	E_R	E_t	E_{pixel}	E_R	E_t	E_{pixel}
Pandey [30]	6.5095	0.1556	9.2	6.1122	0.1282	6.5	5.8059	0.2614	10.3	8.5709	0.2110	6.8	7.4056	0.1467	8.9	6.0923	0.3133	9.1	6.7479	0.2027	8.47
Yuan [31]	1.0494	0.2552	2.8	1.8045	0.6071	1.8	1.4367	0.2423	3.4	2.2242	0.6672	2.6	0.5735	0.2490	2.7	1.7802	1.0525	2.8	1.4781	0.5122	2.68
Koide [24]	0.5397	0.2475	1.2	**0.4886**	0.2309	1.1	0.4454	0.2054	**1.0**	**0.4446**	0.2128	1.4	0.8538	0.2009	1.7	0.7311	0.2107	1.6	0.5839	0.2180	1.33
MFCalib [1]	1.0136	0.2499	3.0	2.1109	0.1789	3.2	3.0187	0.4539	2.8	1.7741	0.5545	1.9	**0.1805**	0.1807	1.2	**0.2175**	0.2892	1.4	1.3859	0.3179	2.25
Ours	**0.4104**	**0.0784**	**0.9**	1.0335	**0.0762**	**0.7**	**0.3354**	**0.0732**	1.3	0.5738	**0.0898**	**0.8**	0.4949	**0.0826**	**0.9**	0.5810	**0.0809**	**0.8**	**0.5715**	**0.0802**	**0.90**

$$E_{\text{pixel}} = \sqrt{\frac{1}{N}\sum_{i=1}^{N}\left\|\pi(\boldsymbol{R}_{\text{true}}\boldsymbol{p}_i^{\mathcal{L}} + \boldsymbol{t}_{\text{true}}) - \pi(\boldsymbol{R}^*\boldsymbol{p}_i^{\mathcal{L}} + \boldsymbol{t}^*)\right\|^2} \quad (35)$$

where $\boldsymbol{p}_i^{\mathcal{L}}$ denotes the i-th LiDAR point, $\pi(\cdot)$ represents the camera projection function parameterized by the intrinsic matrix $\boldsymbol{K}$, and N is the number of valid projected LiDAR points. $[\boldsymbol{R}_{\text{true}}, \boldsymbol{t}_{\text{true}}]$ and $[\boldsymbol{R}^*, \boldsymbol{t}^*]$ denote the ground-truth and estimated extrinsic parameters between the LiDAR and the camera, respectively.

As shown in Table III, compared with the average performance of the current SOTA method Koide [24], our method achieves comparable accuracy in rotation parameter estimation while improving the translation parameter estimation accuracy by 63.2%. Furthermore, our method achieves the lowest reprojection error (0.90 pixels on average, compared to 1.33 pixels of Koide), validating its superior pixel-level alignment in the image plane. This performance stems from two key factors: (1) the abundant geometric edge features extracted using our extraction methods, which are highly sensitive to extrinsic parameters, and (2) the point cloud visibility check module, which effectively mitigates the misinclusion of invisible points in the optimization.

In contrast, Pandey et al. [30], which relies on object reflectivity to perform calibration, performs poorly in practice due to the prevalence of objects with similar materials and distances, limiting its discriminative power. Methods such as Yuan et al. [31] and MFCalib [1], which leverage line features fitted from multiple planes, require abundant planar structures in the scene. Thus, they perform well only in scenes rich with planes, such as Scenes 5 and 6. Koide, which depends on feature matching between dense point cloud projections and camera images, struggles in complex environments due to the sparsity and noise of projected point clouds, increasing the difficulty of feature extraction and matching. Thus, it yields satisfactory results only in simpler scenes like Scenes 1 and 3, but performs inadequately under more challenging conditions.

Based on the above analysis, we conclude that the feature extraction strategy and point cloud visibility check module employed in our method significantly enhance its adaptability to diverse scenarios, improving both the robustness and accuracy of the calibration system.

E. Generalization on Mechanical LiDARs

To evaluate the generalization capability of our method across varying LiDAR scanning patterns and point cloud densities, we conducted additional experiments on the public KITTI dataset [32]. This dataset utilizes a 64-beam mechanical LiDAR (Velodyne HDL-64E), which produces point clouds characterized by a sparse, ring-like distribution. Since our feature extraction module, grounded in local eigenvalue decomposition (Section III-B.1), evaluates spatial distributions within a local neighborhood rather than relying on hardware-specific scanning configurations, it inherently adapts to the rotational scanning mechanism of mechanical LiDARs. Despite this inherent pattern independence, the sparsity of mechanical LiDAR data poses a distinct calibration challenge. Compared to the dense point clouds acquired by solid-state LiDARs (e.g., Livox-AVIA) utilized in our primary experiments, mechanical LiDARs yield substantially sparser point clouds. This sparsity frequently fails to satisfy the minimum neighborhood point requirements necessary for extracting stable 3D geometric edge points.

Nevertheless, our method demonstrates robust adaptability and competitive performance in scenarios containing valid near-field geometric structures. As illustrated in Fig. 19, we selected three representative scenes from the KITTI-00 (002809), KITTI-08 (002996), and KITTI-10 (000590) sequences for evaluation. These scenes feature abundant near-field geometric structures (such as building edges and vehicle contours), which provide sufficient point cloud sampling density to reliably construct high-quality geometric edge points. The comparisons in Fig. 19 demonstrate the projection results before and after calibration. Initially, extrinsic deviations cause evident misalignments between the projected point clouds and the image edges. Following calibration, the point clouds align precisely with their corresponding 2D geometric edges. These results confirm that our method successfully generalizes to sparse mechanical LiDARs. By effectively leveraging near-field structural priors, the algorithm consistently delivers high-precision extrinsic calibration, thereby significantly broadening the hardware applicability of the proposed framework.

F. Running Time Analysis

To evaluate the computational efficiency of the proposed framework, we measured the execution time of its core modules across datasets with varying point densities. Table IV provides a detailed breakdown of the average runtime when processing dense solid-state LiDAR data (≈ 3.3 million points) acquired via a Livox AVIA, compared to sparse mechanical LiDAR data (≈ 130,000 points) from the KITTI dataset.

As presented in Table IV, the edge point extraction module (Section III-B.1) dominates the computational overhead due to intensive KD-tree local searches and eigenvalue decomposition (EVD). However, processing sparse KITTI data takes only 180 ms compared to 8777 ms for dense solid-state LiDAR point clouds, benefiting from significantly smaller search

Authorized licensed use limited to: TONGJI UNIVERSITY. Downloaded on October 18,2026 at 00:47:47 UTC from IEEE Xplore. Restrictions apply.

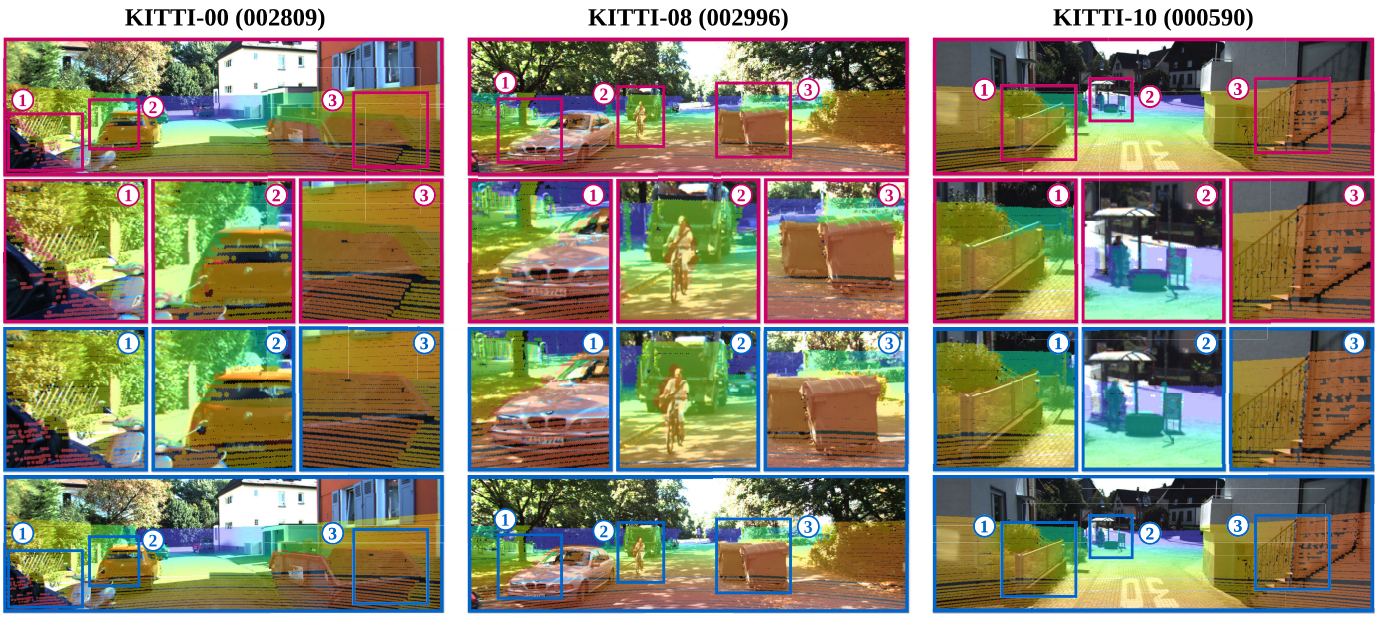


Fig. 19. Qualitative calibration results on the KITTI dataset. Three representative scenes from the KITTI-00 (002809), KITTI-08 (002996), and KITTI-10 (000590) sequences are displayed. The magnified regions (①, ②, and ③) highlight near-field geometric structures. The before- and after-calibration comparisons demonstrate that our method leverages these structures to achieve highly accurate alignment, effectively mitigating the sparsity challenges inherent to mechanical LiDARs.

TABLE IV
RUNNING TIME [MS] ANALYSIS

Point NUM.	Edge Point Extraction		Edge Line Extraction		Point Cloud Visibility Check		Iterative Optimization	
	Local EVD	Edge Extraction	Gradient Scoring	Line Filtering	Spherical Projection	Occlusion Calculation	Feature Matching	Nonlinear Optimization
≈3300000	8777	83406	48	< 0.01	29	47	32	51
≈130000	180	1703	16	< 0.01	1	81	31	22

neighborhoods. During edge line extraction (Section III-B.2), gradient scoring introduces minor computational overhead, whereas the subsequent line filtering stage is exceptionally lightweight, incurring negligible execution time (<0.01 ms). For the visibility check (Section III-C), the occlusion calculation is highly sensitive to the LiDAR's FoV; the narrow FoV of solid-state LiDARs concentrates the points, rendering local spherical searches highly efficient (47 ms), whereas the 360° panoramic FoV of mechanical LiDARs increases spatial indexing complexity (81 ms). Finally, the iterative optimization runtime is primarily governed by the number of valid edge lines and matched points, imposing a higher computational load on the denser point clouds.

In summary, while the framework prioritizes high-precision alignment over strict real-time performance, it maintains a reasonable computational cost. By achieving a robust balance between efficiency and accuracy, it proves well-suited for high-precision extrinsic calibration tasks.

G. Discussion and Limitation

Our proposed calibration framework integrates a monocular depth estimation model (Depth Anything V2) to evaluate and filter geometric image lines. While this effectively mitigates interference from purely textural edges (e.g., lane markings or shadows), it inevitably inherits the fundamental limitations of monocular depth models. Under challenging conditions—such as highly reflective surfaces, drastic illumination changes, or extreme long-range scenes—the depth predictions often become unreliable. Specifically, these data-driven models frequently suffer from boundary blurriness, local shape distortions, and "depth bleeding", where foreground depth values spill into the background.

Despite these flaws, our calibration pipeline remains remarkably robust against depth inaccuracies, largely due to a rigorously designed dual-path feature extraction mechanism. In the first path, geometric edge lines are extracted directly from the depth image (Fig. 7(a)). Since extraction algorithms like LSD require sharp, contiguous gradients, the diffuse and inconsistent gradients caused by depth bleeding and blurriness fail to form continuous lines, naturally eliminating these artifacts early in the pipeline. In the complementary second path, lines are extracted from the high-resolution raw image and subsequently scored using depth gradients. Since precise pixel-level localization relies entirely on texture, the geometric accuracy remains highly resilient to depth boundary bleeding; the depth image serves solely as a geometric confidence filter to discard textural edges. Furthermore, the coarse-to-fine matching strategy provides an additional geometric constraint. Even if spurious edges induced by depth errors pass the 2D filtering stages, they must satisfy strict consistency checks through adaptive elliptical matching and visibility constraints. Without valid LiDAR correspondences, such edges fail to establish cross-modal geometric consistency and are treated

Authorized licensed use limited to: TONGJI UNIVERSITY. Downloaded on October 18,2026 at 00:47:47 UTC from IEEE Xplore. Restrictions apply.

as outliers during optimization, thus having negligible impact on the final calibration.

In addition to depth-related limitations, dynamic objects in real-world driving scenarios may introduce spatial inconsistencies between LiDAR points and image observations due to sensor time offsets. In this work, dynamic instances are not explicitly removed. However, their impact is limited, as the calibration primarily relies on static structural elements (e.g., buildings, road boundaries, and poles), which provide dominant geometric constraints. Furthermore, the visibility-guided formulation and robust optimization suppress inconsistent correspondences. Nevertheless, under rapid motion or significant temporal offsets, residual errors may persist. Incorporating explicit dynamic object filtering or motion-aware modeling could further improve robustness and is left for future work.

In terms of scalability, the current work focuses on a single LiDAR-camera pair. Nevertheless, the proposed geometric edge constraint can be naturally extended to multi-camera and multi-LiDAR systems. For each LiDAR-camera pair with sufficient FoV overlap, the same point-to-line reprojection constraint can be constructed and further integrated into a sensor graph, where sensor poses are treated as nodes and cross-modal geometric constraints act as edges. Practically, this extension relies on sufficient co-visible structures. In cases where direct FoV overlap is unavailable, indirect constraints through intermediate sensors or auxiliary kinematic constraints can be introduced.

Finally, the superior calibration accuracy of the proposed method comes at the cost of increased computational overhead compared to lightweight baselines. This is primarily due to the edge point extraction module, which involves complex geometric reasoning and consistency checks. While this design is well-suited for offline calibration and high-precision applications, it may limit deployment in real-time or resource-constrained scenarios. Future work will focus on improving efficiency through correspondence pruning, more efficient optimization, and learning-based priors for faster convergence.

Overall, the proposed framework emphasizes geometric reliability and calibration accuracy through multi-stage consistency verification, effectively mitigating the impacts of depth uncertainty and spurious correspondences, while acknowledging the remaining challenges in dynamic environments, computational efficiency, and extension to more general multi-sensor configurations.

V. Conclusion

In this paper, we presented a novel target-free method for the automatic extrinsic calibration of LiDAR-camera systems. The core of our method lies in robust multi-modal feature extraction and matching. For point clouds, we introduced eigenvector-based detectors to identify geometric edge points. For images, we developed a method that fuses learning-based depth estimation with traditional line extraction to extract robust geometric edge lines. Additionally, a visibility check module based on the overlap of Gaussian ellipsoids and local normal vector consistency was proposed to eliminate occluded points. This module prevents invalid features from degrading the optimization and enhances calibration accuracy. Comprehensive experimental results demonstrate that our method outperforms existing techniques in terms of both robustness and calibration accuracy.

In future work, we will focus on enhancing the framework's generalization capabilities for mainstream mechanical LiDAR systems. Building upon its proven efficacy in structure-rich environments, we aim to design more resilient feature extraction strategies tailored for extremely sparse and low-resolution point clouds, thereby enabling accurate calibration across a wider range of hardware configurations.

References

[1] T. Ye, W. Xu, C. Zheng, and Y. Cui, "MFCalib: Single-shot and automatic extrinsic calibration for LiDAR and camera in targetless environments based on multi-feature edge," in *Proc. IEEE/RSJ Int. Conf. Intell. Robots Syst. (IROS)*, Oct. 2024, pp. 864–871.

[2] R. She et al., "RobustMat: Neural diffusion for street landmark patch matching under challenging environments," *IEEE Trans. Image Process.*, vol. 32, pp. 5550–5563, 2023.

[3] Z. Wang, J. Yi, A. Chen, and G. Han, "MoVis: When 3D object detection is like human monocular vision," *IEEE Trans. Image Process.*, vol. 34, pp. 3025–3040, 2025.

[4] C. Huang, T. He, H. Ren, W. Wang, B. Lin, and D. Cai, "OBMO: One bounding box multiple objects for monocular 3D object detection," *IEEE Trans. Image Process.*, vol. 32, pp. 6570–6581, 2023.

[5] H. Liu, H. Liu, Y. Wang, F. Sun, and W. Huang, "Fine-grained multilevel fusion for anti-occlusion monocular 3D object detection," *IEEE Trans. Image Process.*, vol. 31, pp. 4050–4061, 2022.

[6] T. Sun, Z. Zhang, X. Tan, Y. Qu, and Y. Xie, "Image understands point cloud: Weakly supervised 3D semantic segmentation via association learning," *IEEE Trans. Image Process.*, vol. 33, pp. 1838–1852, 2024.

[7] Y. Wu, R. Li, Z. Qin, X. Zhao, and X. Li, "HeightFormer: Explicit height modeling without extra data for camera-only 3D object detection in bird's eye view," *IEEE Trans. Image Process.*, vol. 34, pp. 689–700, 2025.

[8] Y. Sun, J. Li, Y. Wang, X. Xu, X. Yang, and Z. Sun, "ATOP: An attention-to-optimization approach for automatic LiDAR-camera calibration via cross-modal object matching," *IEEE Trans. Intell. Vehicles*, vol. 8, no. 1, pp. 696–708, Jan. 2023.

[9] G. Iyer, R. K. Ram, J. K. Murthy, and K. M. Krishna, "CalibNet: Geometrically supervised extrinsic calibration using 3D spatial transformer networks," in *Proc. IEEE/RSJ Int. Conf. Intell. Robots Syst. (IROS)*, Oct. 2018, pp. 1110–1117.

[10] R. G. von Gioi, J. Jakubowicz, J.-M. Morel, and G. Randall, "LSD: A fast line segment detector with a false detection control," *IEEE Trans. Pattern Anal. Mach. Intell.*, vol. 32, no. 4, pp. 722–732, Apr. 2010.

[11] J. Canny, "A computational approach to edge detection," *IEEE Trans. Pattern Anal. Mach. Intell.*, vols. PAMI-8, no. 6, pp. 679–698, Nov. 1986.

[12] R. Ishikawa, S. Zhou, Y. Sato, T. Oishi, and K. Ikeuchi, "LiDAR-camera calibration using intensity variance cost," in *Proc. IEEE Int. Conf. Robot. Autom. (ICRA)*, May 2024, pp. 10688–10694.

[13] Y. Dong, Y. Liu, L. Li, H. Deng, J. Tang, and J. Li, "Automatic LiDAR–camera extrinsic calibration using pseudoimage and multiple targets," *IEEE Trans. Instrum. Meas.*, vol. 73, pp. 1–13, 2024.

[14] Q. Zhang and R. Pless, "Extrinsic calibration of a camera and laser range finder (improves camera calibration)," in *Proc. IEEE/RSJ Int. Conf. Intell. Robots Syst. (IROS)*, vol. 3, Feb. 2004, pp. 2301–2306.

[15] Y. Xie et al., "A4LidarTag: Depth-based fiducial marker for extrinsic calibration of solid-state LiDAR and camera," *IEEE Robot. Autom. Lett.*, vol. 7, no. 3, pp. 6487–6494, Jul. 2022.

[16] X. Li, F. He, S. Li, Y. Zhou, C. Xia, and X. Wang, "Accurate and automatic extrinsic calibration for a monocular camera and heterogenous 3D LiDARs," *IEEE Sensors J.*, vol. 22, no. 16, pp. 16472–16480, Aug. 2022.

[17] J. Cui, J. Niu, Y. He, D. Liu, and Z. Ouyang, "ACLC: Automatic calibration for nonrepetitive scanning LiDAR-camera system based on point cloud noise optimization," *IEEE Trans. Instrum. Meas.*, vol. 73, pp. 1–14, 2024.

[18] M. Wei, J.-G. Lu, N. Ye, Z. Zhu, Q. Zhang, and Y. Wang, "A full-lifecycle calibration method for camera and LiDAR in autonomous driving," *IEEE Trans. Instrum. Meas.*, vol. 74, pp. 1–17, 2025.

Authorized licensed use limited to: TONGJI UNIVERSITY. Downloaded on October 18,2026 at 00:47:47 UTC from IEEE Xplore. Restrictions apply.

[19] D. Wang, B. Liu, Z. Liu, J. Wang, M. Xu, and L. Kong, "LONS-calib: Extrinsic calibration of LiDAR and omnidirectional camera with negative space," *IEEE Trans. Instrum. Meas.*, vol. 74, pp. 1–13, 2025.
[20] L. Zhou, Y. Zhang, L. Wang, and J. Zhang, "Targetless extrinsic calibration of camera and LiDAR via identifying true 2-D–3-D line matching among unknown line correspondences of structured environments," *IEEE Trans. Instrum. Meas.*, vol. 73, pp. 1–17, 2024.
[21] X. Liu, C. Yuan, and F. Zhang, "Targetless extrinsic calibration of multiple small FoV LiDARs and cameras using adaptive voxelization," *IEEE Trans. Instrum. Meas.*, vol. 71, pp. 1–12, 2022.
[22] T. Fu, H. Yu, W. Yang, Y. Hu, and S. Scherer, "Targetless extrinsic calibration of stereo, thermal, and laser sensors in structured environments," *IEEE Trans. Instrum. Meas.*, vol. 71, pp. 1–11, 2022.
[23] N. Ou, H. Cai, and J. Wang, "Targetless LiDAR-camera calibration via cross-modality structure consistency," *IEEE Trans. Intell. Vehicles*, vol. 9, no. 1, pp. 2636–2648, Jan. 2024.
[24] K. Koide, S. Oishi, M. Yokozuka, and A. Banno, "General, single-shot, target-less, and automatic LiDAR-camera extrinsic calibration toolbox," in *Proc. IEEE Int. Conf. Robot. Autom. (ICRA)*, May 2023, pp. 11301–11307.
[25] S. Zhou, S. Xie, R. Ishikawa, and T. Oishi, "Robust LiDAR-camera calibration with 2D Gaussian splatting," *IEEE Robot. Autom. Lett.*, vol. 10, no. 5, pp. 4674–4681, May 2025.
[26] Q. Guo, C. Zhang, Y. Zhang, and H. Liu, "An efficient SVD-based method for image denoising," *IEEE Trans. Circuits Syst. Video Technol.*, vol. 26, no. 5, pp. 868–880, May 2016.
[27] L. Cao, "Singular value decomposition applied to digital image processing," Arizona State Univ. Polytech. Campus, Mesa, AZ, USA, Tech. Rep., 2006, pp. 1–15.
[28] L. Yang et al., "Depth anything v2," in *Proc. Adv. Neural Inf. Process. Syst. (NeurIPS)*, vol. 37, Mar. 2024, pp. 21875–21911.
[29] J. E. Bresenham, "Algorithm for computer control of a digital plotter," *IBM Syst. J.*, vol. 4, no. 1, pp. 25–30, Mar. 1965.
[30] G. Pandey, J. McBride, S. Savarese, and R. Eustice, "Automatic targetless extrinsic calibration of a 3D LiDAR and camera by maximizing mutual information," in *Proc. AAAI Conf. Artif. Intell.*, Sep. 2021, pp. 2053–2059.
[31] C. Yuan, X. Liu, X. Hong, and F. Zhang, "Pixel-level extrinsic self calibration of high resolution LiDAR and camera in targetless environments," *IEEE Robot. Autom. Lett.*, vol. 6, no. 4, pp. 7517–7524, Oct. 2021.
[32] A. Geiger, P. Lenz, C. Stiller, and R. Urtasun, "Vision meets robotics: The KITTI dataset," *Int. J. Robot. Res.*, vol. 32, no. 11, pp. 1231–1237, Sep. 2013.



Baosheng Zhang received the B.S. degree in automation from Dalian Minzu University, Dalian, China, in 2019, and the M.S. degree in control science and engineering from Northeastern University, Shenyang, China, in 2022. He is currently pursuing the Ph.D. degree with Tongji University. His research interests include visual and LiDAR SLAM, and autonomous intelligent systems.



Lin Zhang (Senior Member, IEEE) received the B.Sc. and M.Sc. degrees from the Department of Computer Science and Engineering, Shanghai Jiao Tong University, Shanghai, China, in 2003 and 2006, respectively, and the Ph.D. degree from the Department of Computing, The Hong Kong Polytechnic University, Hong Kong, in 2011. From March 2011 to August 2011, he was a Research Associate with the Department of Computing, The Hong Kong Polytechnic University. In August 2011, he joined the School of Software Engineering, Tongji University, Shanghai, where he is currently a Full Professor with the School of Computer Science and Technology. His current research interests include machine vision and unmanned intelligent systems. He serves as an Associate Editor for IEEE ROBOTICS AND AUTOMATION LETTERS and *Journal of Visual Communication and Image Representation*. He was awarded as a Young Scholar of Changjiang Scholars Program, Ministry of Education, China.



Shengjie Zhao (Senior Member, IEEE) received the B.S. degree in electrical engineering from the University of Science and Technology of China, Hefei, China, in 1988, the M.S. degree in electrical and computer engineering from China Aerospace Institute, Beijing, China, in 1991, and the Ph.D. degree in electrical and computer engineering from Texas A&M University, College Station, TX, USA, in 2004. He is currently a Professor with the School of Computer Science and Technology and the College of Electronics and Information Engineering, Tongji University, Shanghai, China. In previous postings, he conducted research at Lucent Technologies, Whippany, NJ, USA, and China Aerospace Science and Industry Corporation, Beijing. He is a fellow of the Thousand Talents Program of China and an Academician of the International Eurasian Academy of Sciences. His research interests include artificial intelligence, big data, wireless communications, image processing, and signal processing.



Yicong Zhou (Senior Member, IEEE) received the B.S. degree in electrical engineering from Hunan University, Changsha, China, and the M.S. and Ph.D. degrees in electrical engineering from Tufts University, Medford, MA, USA. He is currently a Full Professor and the Director of the Vision and Image Processing Laboratory, Department of Computer and Information Science, University of Macau, Macau, China. His research interests include chaotic systems, multimedia security, computer vision, and machine learning. He was a recipient of the Third Price of Macau Natural Science Award in 2014. He serves as an Associate Editor for *Neurocomputing*, *Journal of Visual Communication and Image Representation*, and *Signal Processing: Image Communication*. He is the Co-Chair of the Technical Committee on Cognitive Computing in the IEEE Systems, Man, and Cybernetics Society. He is a Senior Member of the International Society for Optical Engineering (SPIE).

Authorized licensed use limited to: TONGJI UNIVERSITY. Downloaded on October 18,2026 at 00:47:47 UTC from IEEE Xplore. Restrictions apply.